Recursion

Fundamentos

- *La recursión se presenta cuando en la definición de algo interviene lo que que-remos definir.*
- *Si nos ubicamos en la otra vereda, se dice que cuando queremos definir algo no puede intervenir como parte de la definición lo que estamos intentando definir.*
- *¡Excepto que hablemos de recursión!*

Ejemplos de Recursión

- ***Un ramo de rosas***

  Es o bien una rosa, o una rosa con un ramo de rosas.
- ***Los descendientes del Sr. Pérez***

  Es o bien un hijo del Sr. Pérez o un hijo de un descendiente del Sr. Pérez.
- *Como vemos en ambas definiciones interviene el objeto definido, esto gramaticalmente sería incorrecto.*

Recursión Gráfica

- *Aparece cuando el cuadro general está dentro de la figura representada.*
- *Ejemplos:*

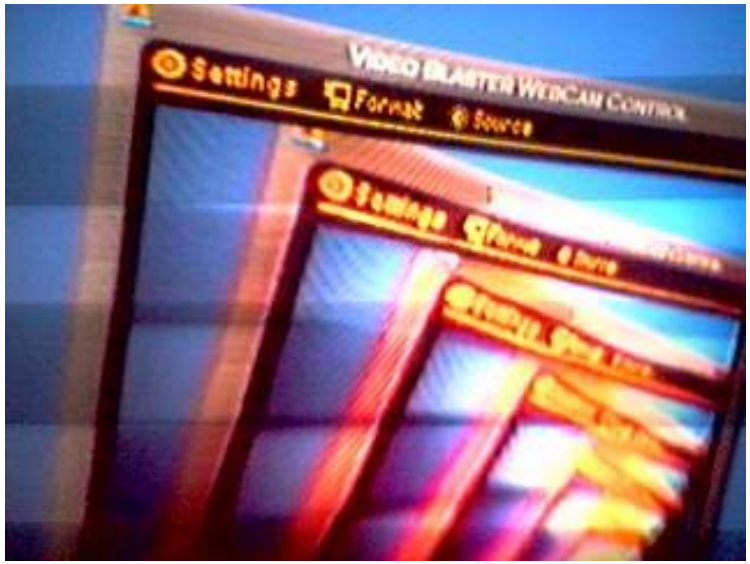
  Cuando la cámara de TV enfoca a un monitor.

  Una lata de polvo de hornear Royal.

  Todo recipiente que en su etiqueta esté representado el mismo recipiente.

Recursión Gráfica





Recursión Gráfica





Recursión Gráfica

- *En los casos vistos, tenemos recursión infinita, esto es, vemos solo dos o tres monitores o dos o tres latas por una cuestión de resolución.*
- *Conceptualmente hay infinitos monitores e infinitas latas.*
- *Para nuestros fines nos interesa una recursión controlada, que no sea infinita.*

Recursión

- *Todos nuestros proyectos van a terminar en un algoritmo y la definición de algoritmo dice que es:*

  Un procedimiento para resolver un problema en un número finito de pasos.

- *Esto nos obliga a descartar todos los procedimientos que no se resuelven en un número finito de pasos.*

Algoritmo Recursivo

- *Todo algoritmo recursivo debe tener una **condición de salida** y una **parte recursiva** propiamente dicha.*
- *Se hace evidente a partir de esta definición que algo debe variar en los argumentos de la función entre llamada y llamada.*
- *Es la única garantía de que la recursión no sea infinita.*

Problemas Recursivos

- *Los problemas que se pretende resolver mediante recursión deben ser naturalmente recursivos, p. Ej.:*
  - *Factorial de un número*
  - *Sucesión de Fibonaci*
  - *Cálculo del MCD por el método de Euclides*
  - *Ordenamiento Quick Sort*
  - *Torres de Hanoi*

Ventajas de la Recursión

- *Se obtienen versiones de algoritmos más claras y sencillas.*
- *Las funciones recursivas son muy elegantes en términos de expresión.*
- *Sintetizan en muy poco espacio lo que de otra forma sería muy complicado expresar en forma iterativa.*
- *Se adaptan en forma natural a problemas inherentemente recursivos.*

Desventajas de la Recursión

- *Consumen más memoria y bajo circunstancias muy adversas pueden llegar a agotar la memoria.*
- *Son un poco más lentas de ejecutar que las versiones iterativas por la gran cantidad de llamadas a funciones.*
- *Son un poco más difíciles de comprender, pero una vez entendidas, la solución es más simple.*

Funcionamiento

- *Cuando una función se llama a sí misma, se asigna espacio en la pila para las nuevas variables locales y parámetros.*
- *El código de la función se ejecuta con las nuevas variables desde el principio.*
- *Una llamada recursiva no hace una nueva copia de la función. Son solo nuevos argumentos.*

Funcionamiento

- *Al volver de una llamada recursiva se recuperan de la pila las variables locales y los parámetros antiguos.*
- *La ejecución se reanuda en la línea posterior a la llamada de la función dentro de la función.*
- *Podría decirse que las funciones recursivas tienen un efecto telescópico que las lleva fuera y dentro de si mismas.*

Factorial

- *El factorial se define en forma recursiva como:*

```
n!   1          si n = 0
     n*(n-1)!   si n > 0
```

- *La primera línea representa la **condición de salida**.*
- *La segunda línea es la **parte recursiva propiamente dicha**.*

Sucesión de Fibonaci

- *La sucesión de Fibonaci se define así:*

Fib(n)

```
1                          si  n=0, n=1
Fib(n-1)+Fib(n-2)          si  n > 1
```

- *La primera línea representa la **condición de salida**.*
- *La segunda línea es la **parte recursiva propiamente dicha**.*

Algoritmo de Euclides

- *El algoritmo de Euclides permite calcular el Máximo Común Divisor de dos números enteros.*

mcd(m, n)

Si m%n = 0	mcd = n
Si-no	resto = m%n mcd = mcd(n,resto)

- *Primera y segunda línea son **condición de salida** y **parte recursiva.***

Algoritmo de Euclides

- *Por ejemplo si tengo los números 20 y 72*

mcd(20, 72)

```
m%n <> 0    por lo que
    resto = 20%72  da 20
    ahora llamamos la función con
    m=72 n=20
```

Algoritmo de Euclides

mcd(72, 20)

m%n <> 0 por lo que
resto = 72%20 da 12
ahora llamamos la función con
m=20 n=12

mcd(20, 12)

m%n <> 0 por lo que
resto = 20%12 da 8
ahora llamamos la función con
m=12 n=8

Algoritmo de Euclides

mcd(12, 8)

m%n <> 0 por lo que
resto = 12%8 da 4
ahora llamamos la función con
m=8 n=4

mcd(8, 4)

m%n = 0 por lo que el mcd da

mcd = 4

Algoritmo de Euclides

- *En este último paso ha actuado la condición de salida.*
- *En todos los pasos anteriores había actuado la parte recursiva propiamente dicha.*
- *La resolución de cada llamada a función se produce cuando se comienzan a cerrar las llamadas a partir de ejecutarse la condición de salida.*

```
#include <stdio.h>
void main()
{
  int a, b;

  printf( "\n\n\t\tCálculo del MCD por
  Euclides\n\n" );
  printf( "Ingrese los dos números: " );
  scanf( "%d %d", &a, &b );
  printf( "El MCD de %d y %d es: %d\n",
  a, b, MCD( a, b ) );
}

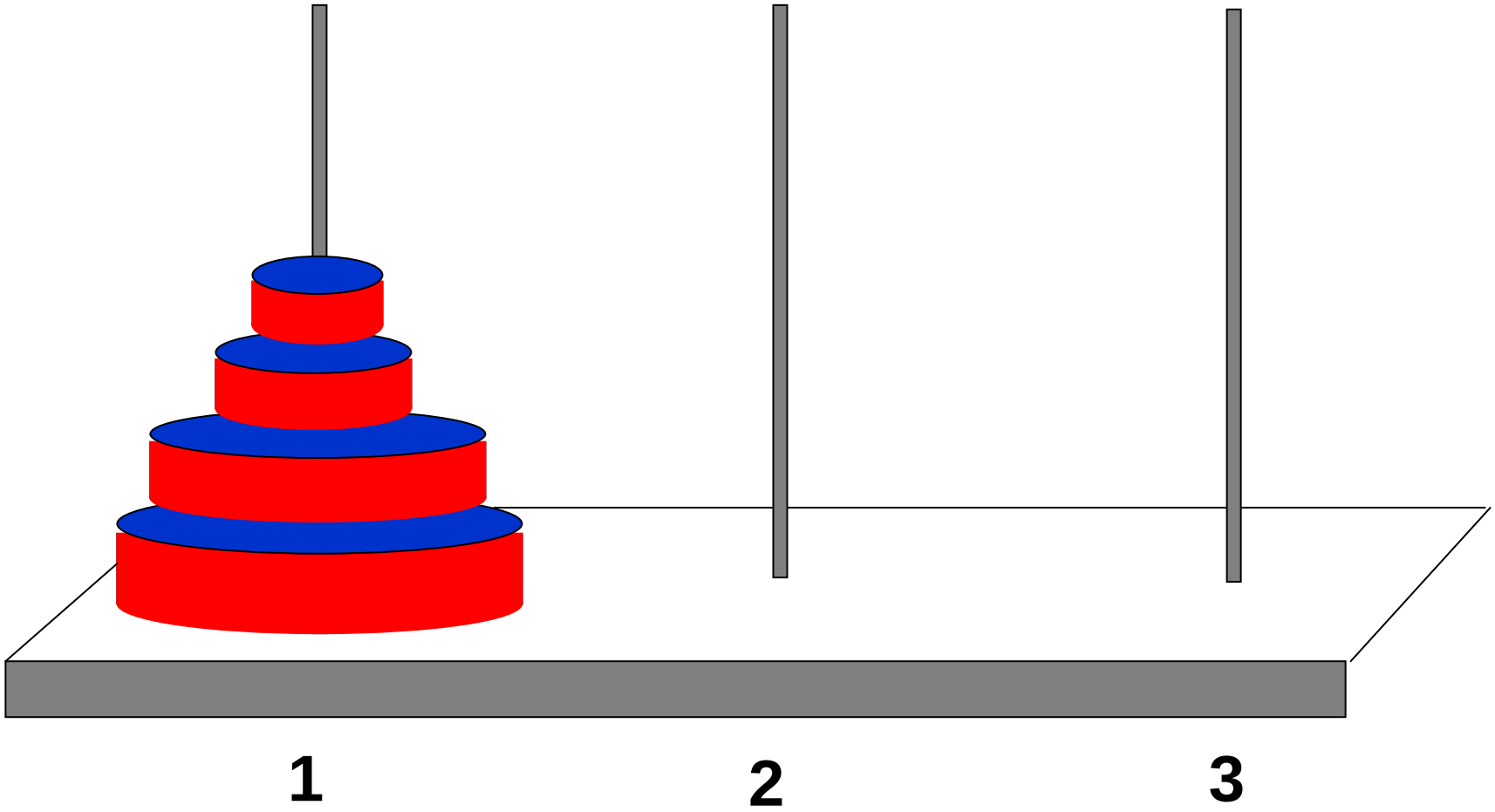
int MCD( int num, int den ) {
  if( num%den == 0 ) return den;
  else return( MCD( den, num%den ) );
}
```

Torres de Hanoi

- *Torres de Hanoi es un juego muy antiguo que consiste en transportar discos desde un eje a otro (utilizando un eje auxiliar) de a uno por vez, sin que jamás un disco de mayor superficie quede por encima de un disco de menor superficie.*

- *El conjunto de discos apilados es lo que forma la "torre".*

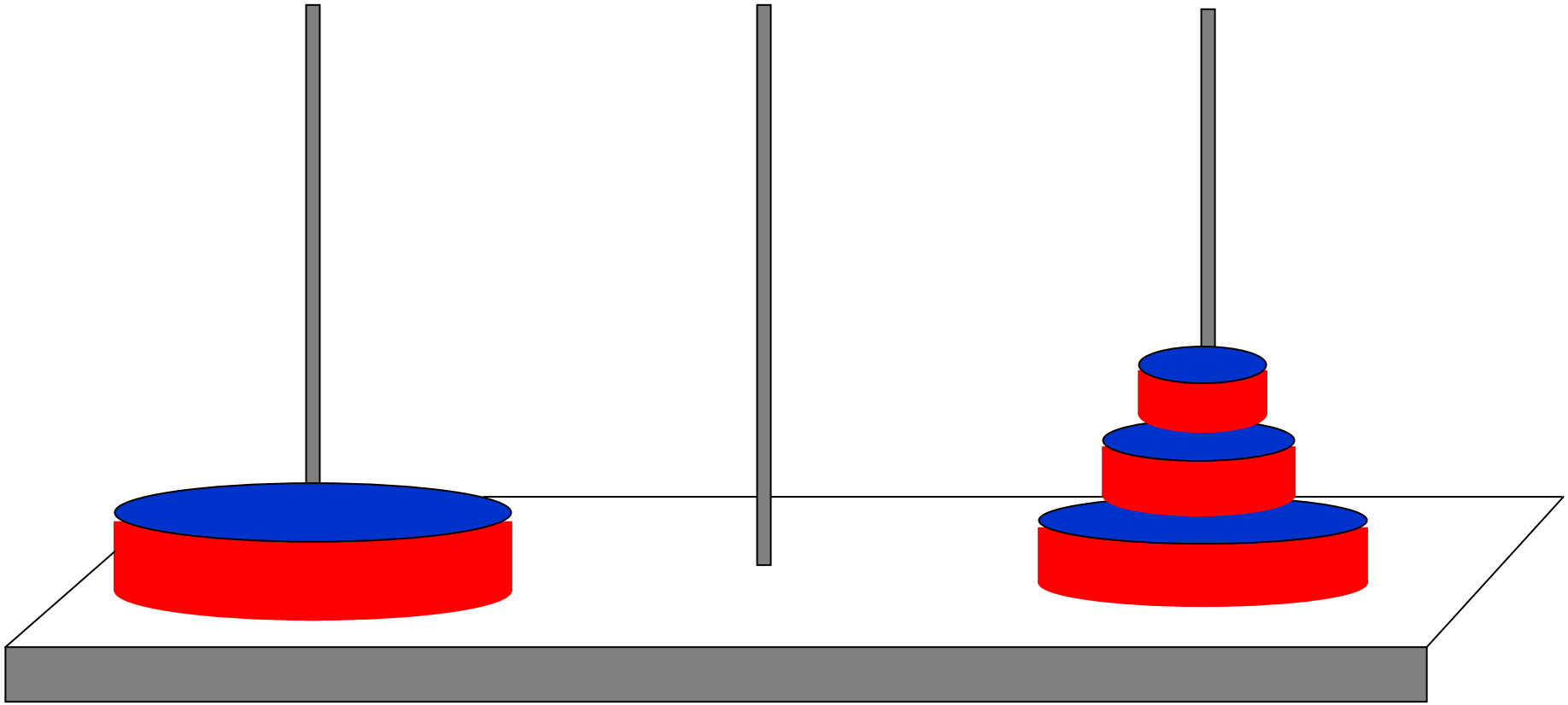
Torres de Hanoi



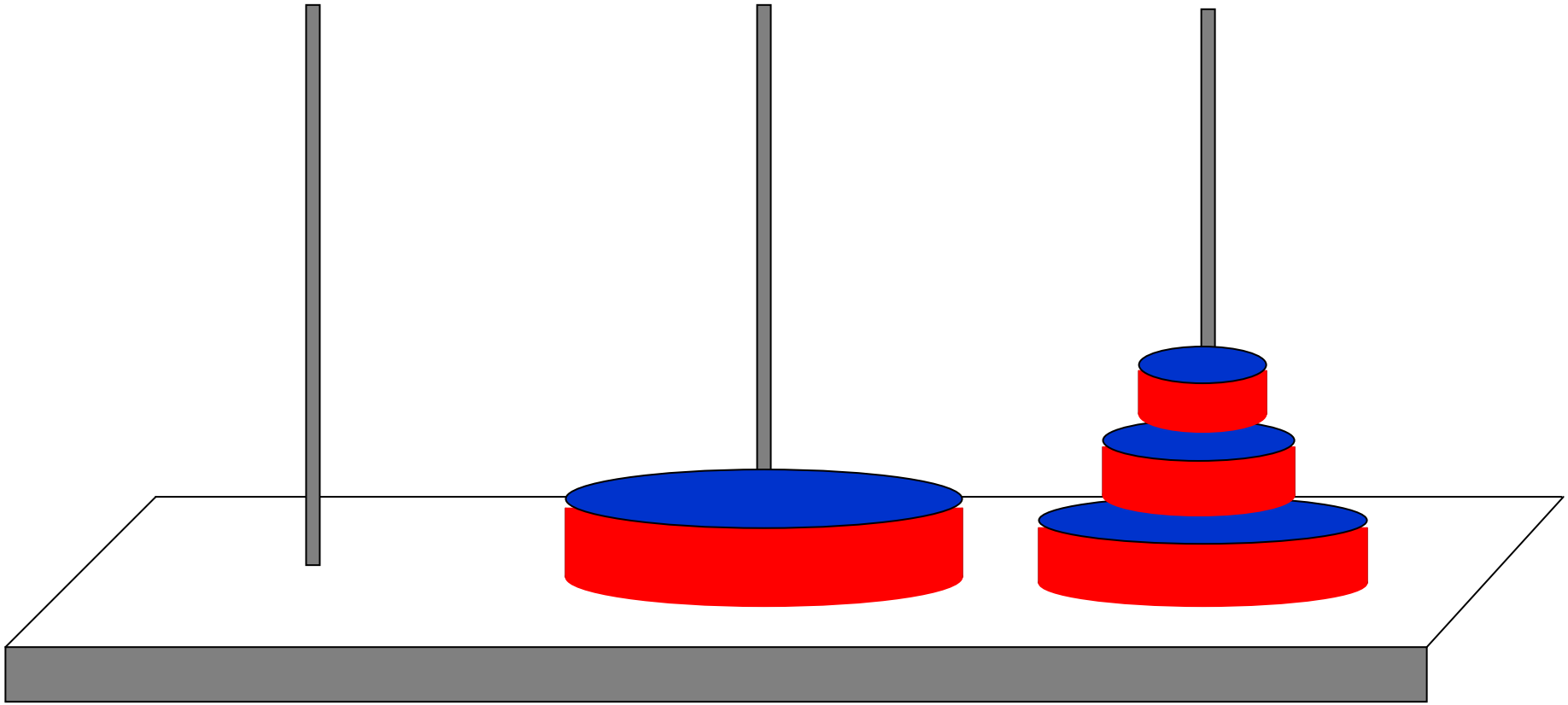
Torres de Hanoi

- *Pretendemos desplazar los discos del eje 1 al eje 2 utilizando el eje 3 como auxiliar.*
- *Este problema consiste en pasar una torre de altura 3 al eje 3, el disco mayor desde el eje 1 al 2 y por último llevar la torre de 3 discos del eje 3 (auxiliar) al eje 2.*
- *El llevar “torres” implica descomponer.*

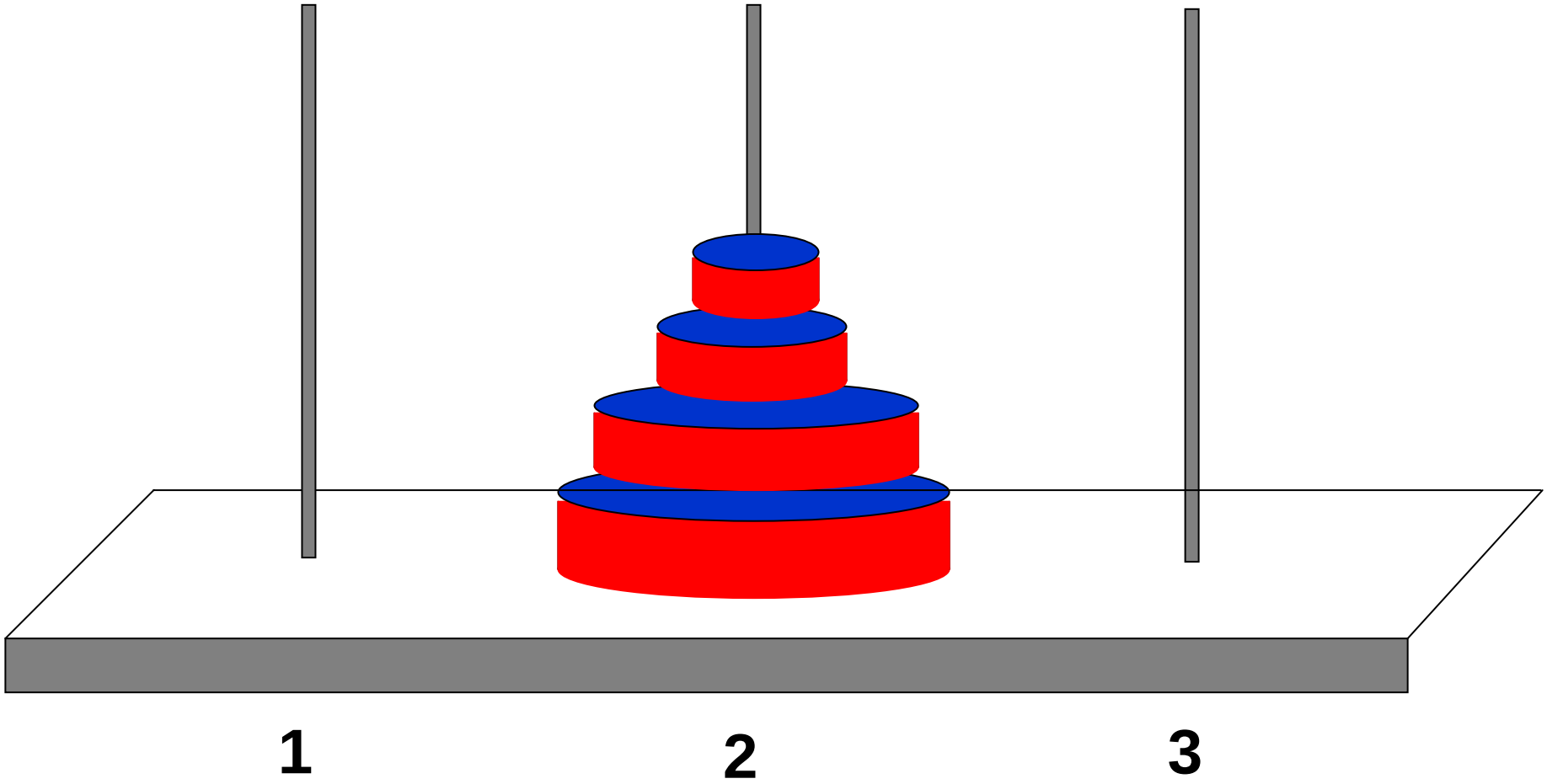
Torres de Hanoi



Torres de Hanoi



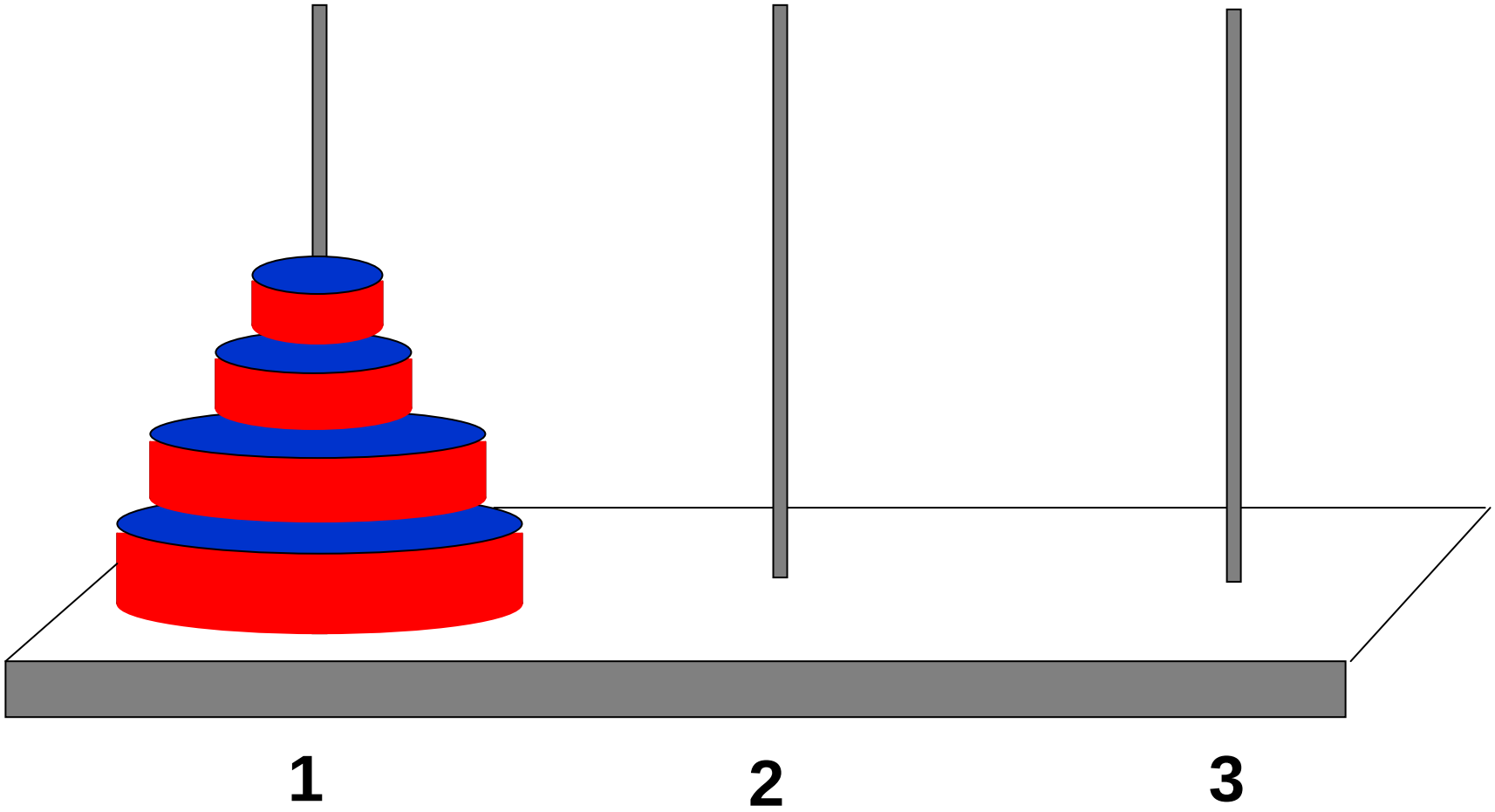
Torres de Hanoi



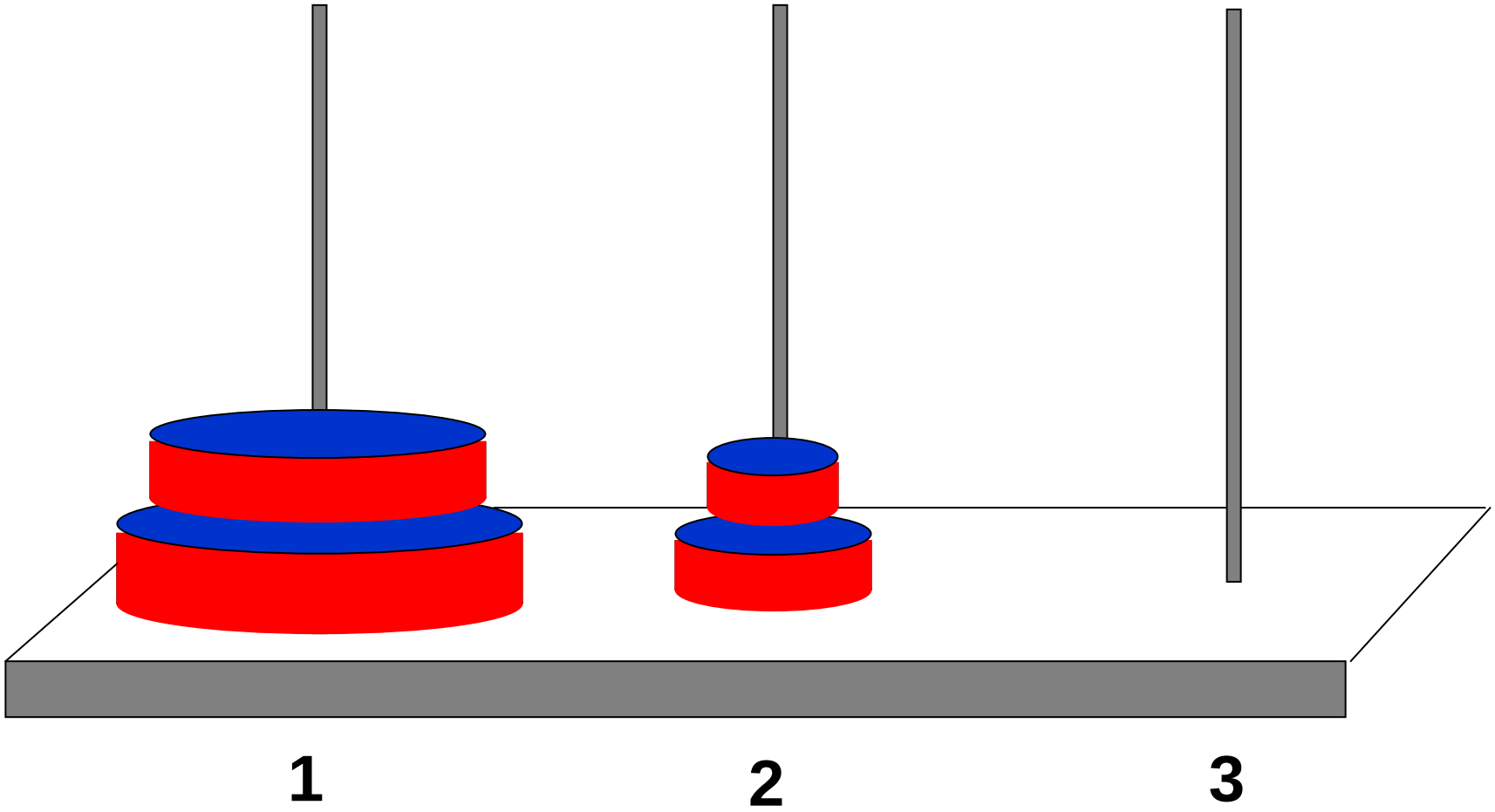
Torres de Hanoi

- *El primer paso de llevar una torre de altura 3 de los ejes 1 a 3, se descompone en:*
- *Llevar una torre de altura 2 del eje 1 al eje 2, llevar el disco restante del eje 1 al 3 y por último llevar la torre de altura 2 del eje auxiliar (2) al eje final 3.*
- *Comprobar que origen, destino y auxiliar van cambiando en los movimientos.*

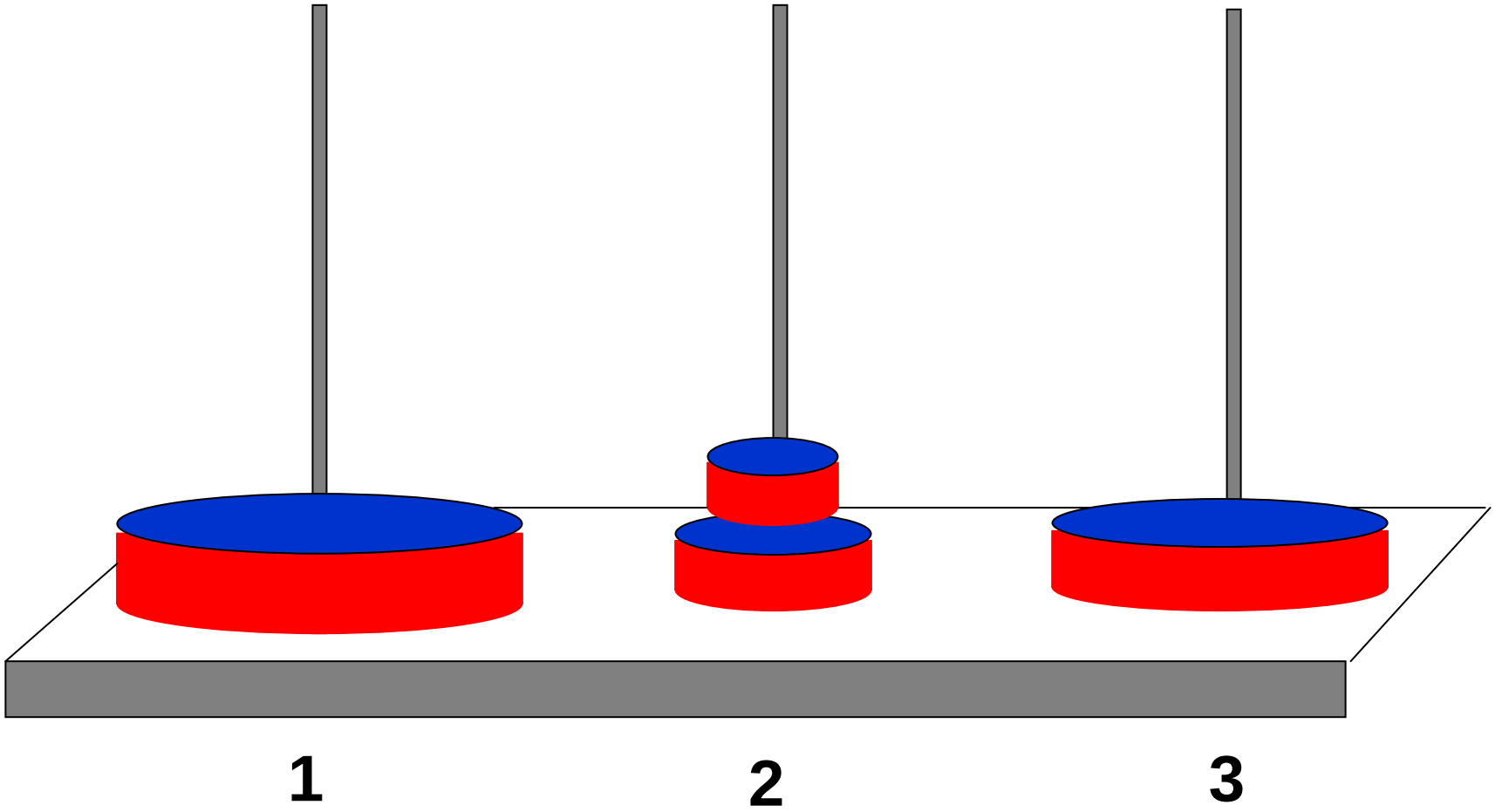
Traslado de la torre de 3



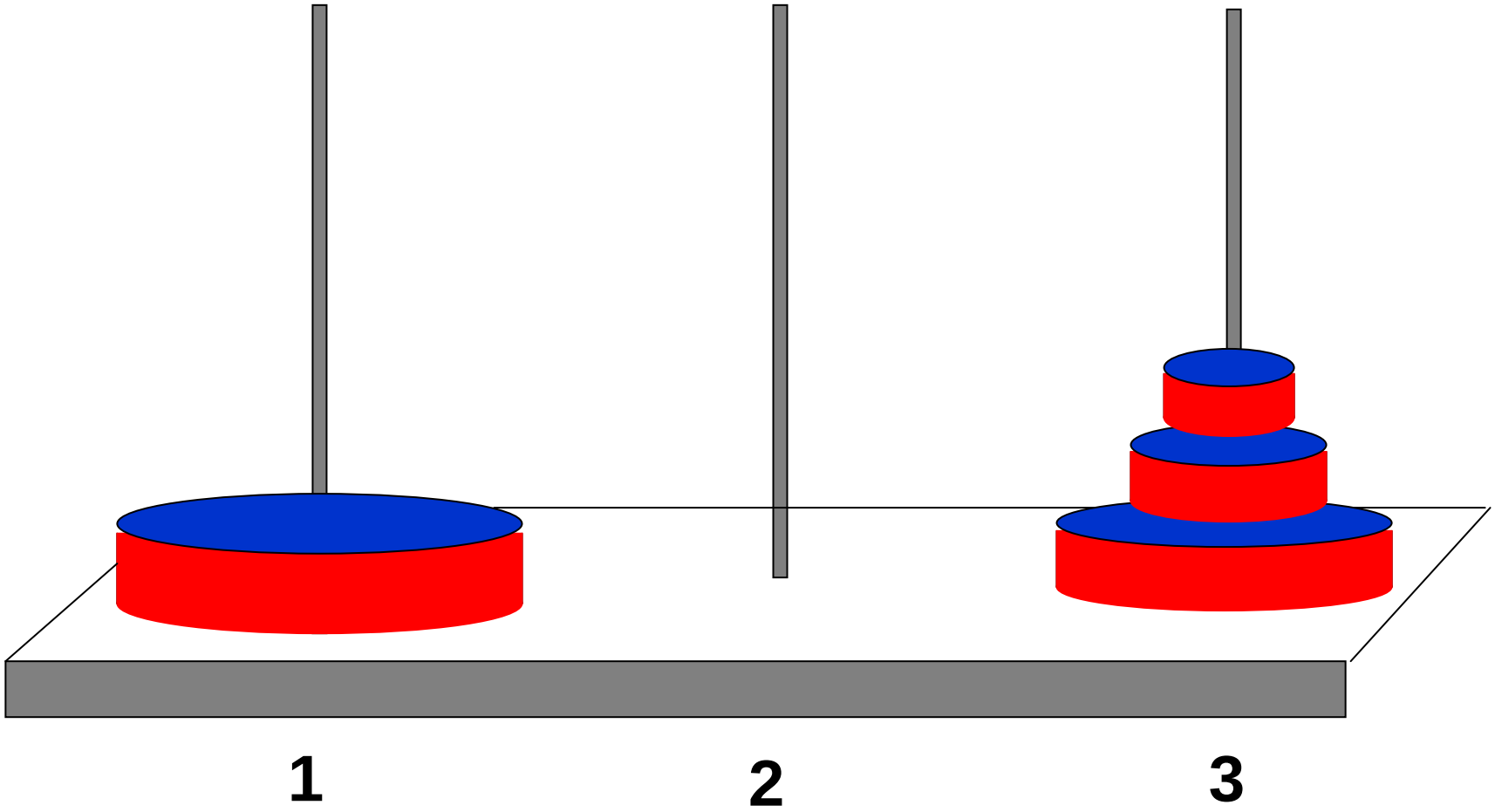
Traslado de la torre de 3



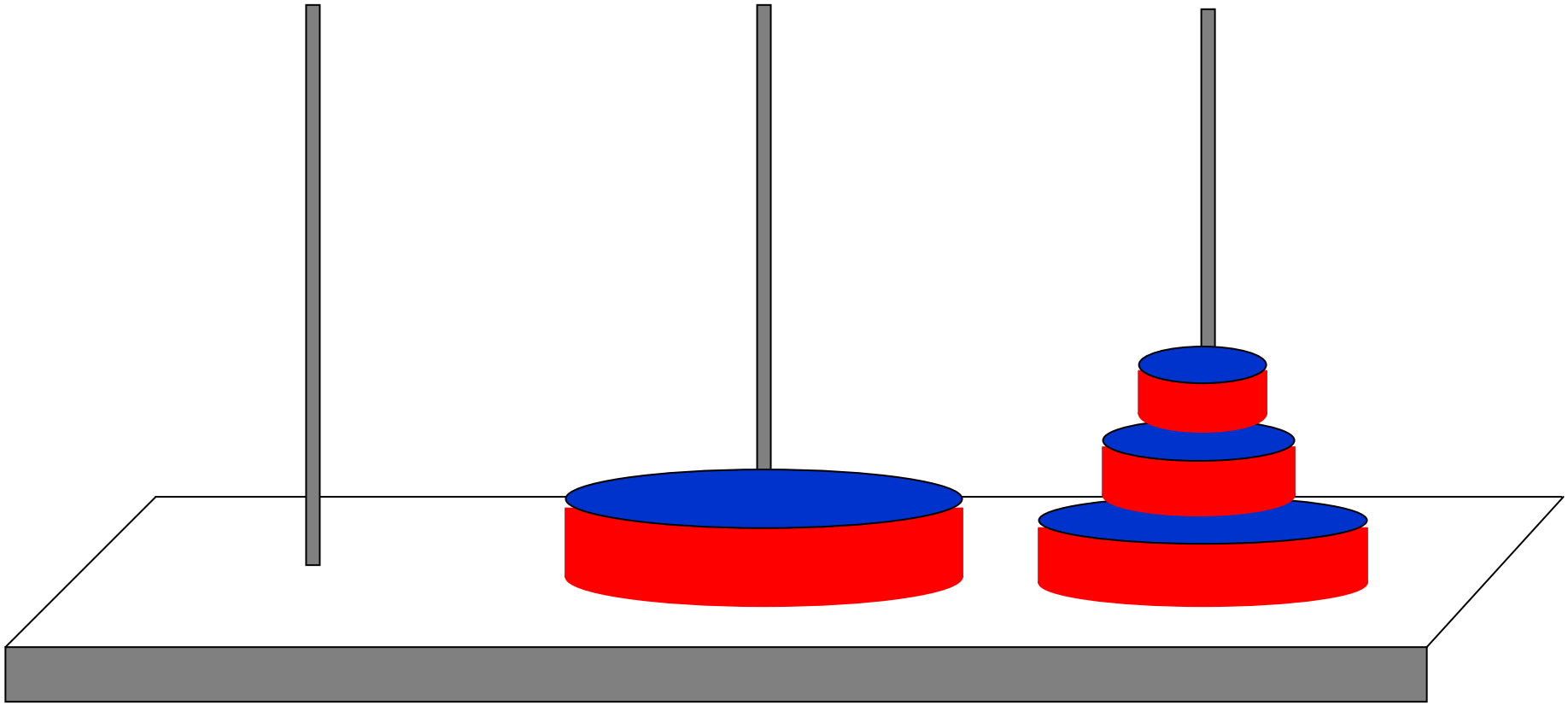
Traslado de la torre de 3



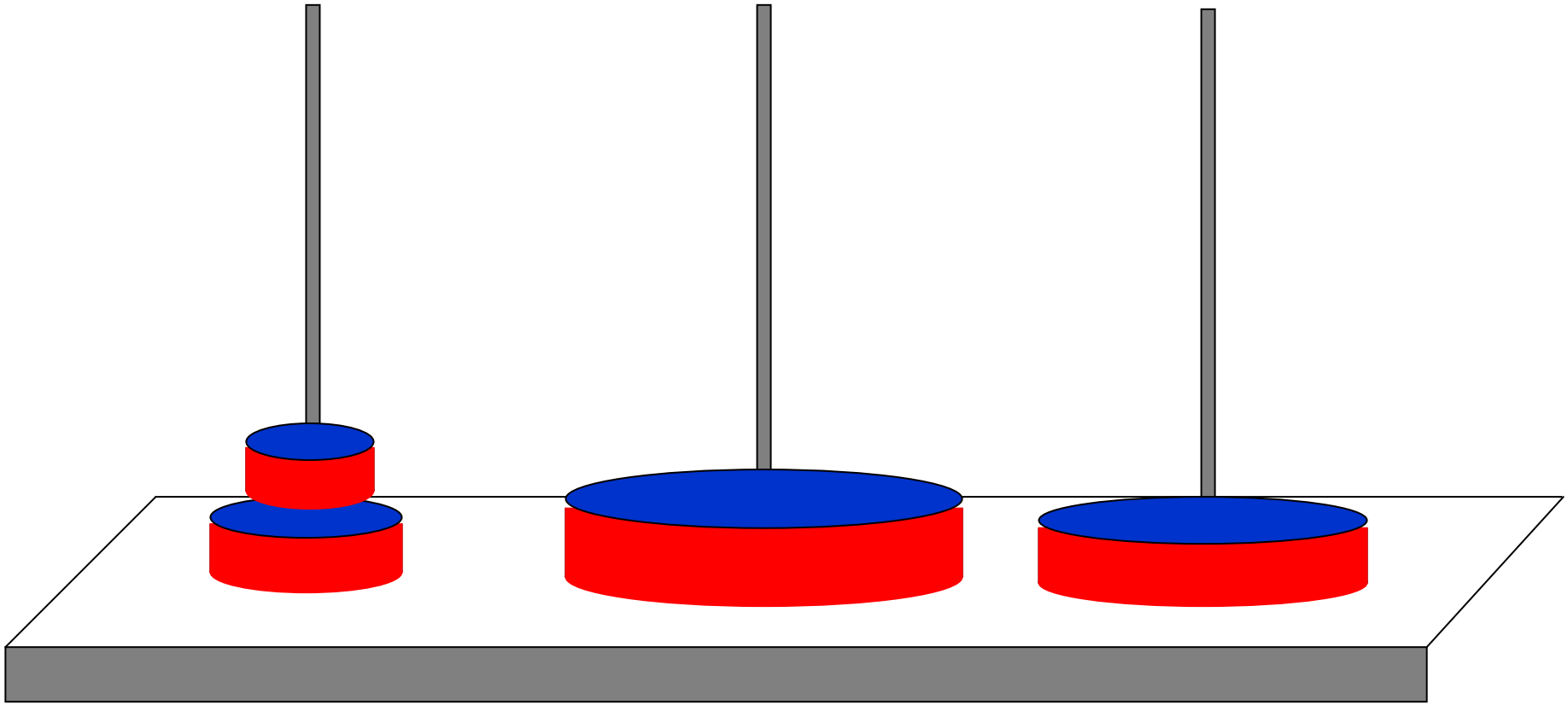
Traslado de la torre de 3



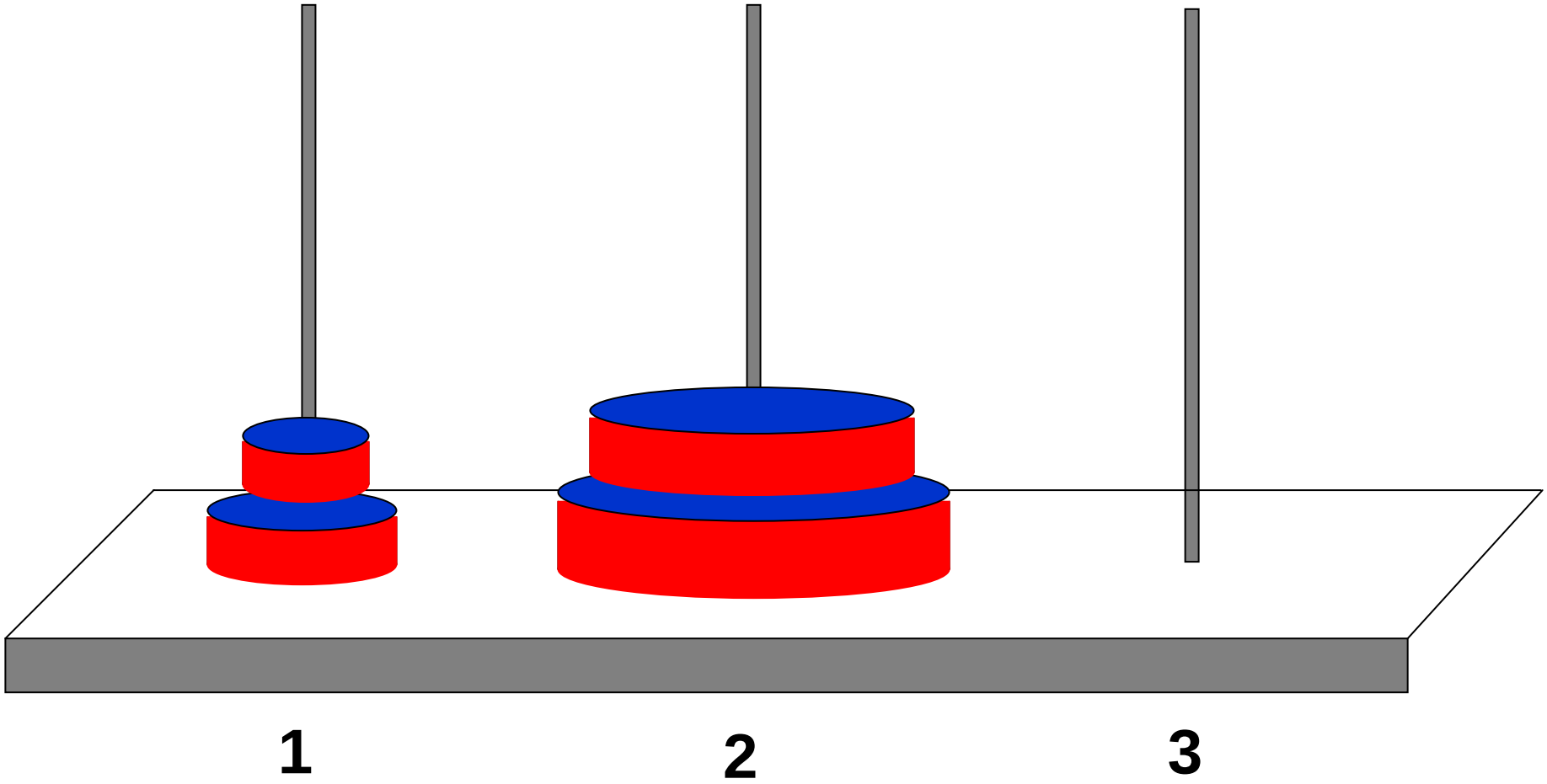
Torres de Hanoi



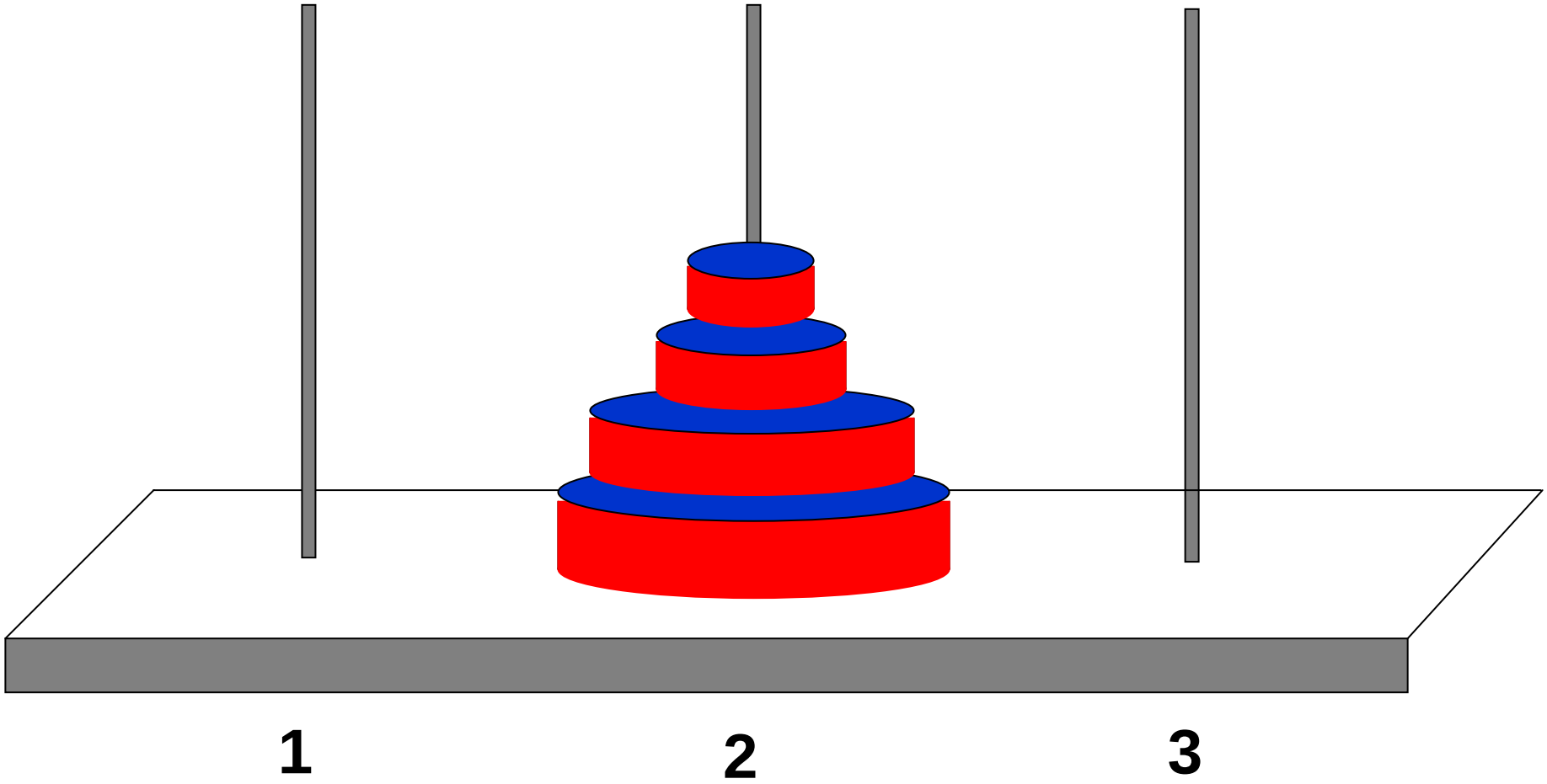
Torres de Hanoi



Torres de Hanoi



Torres de Hanoi



Algoritmo de Hanoi

- *Para el primer movimiento el eje **1** es el origen (**o**), el eje **2** es el destino (**d**) y el eje **3** es el intermedio (**i**).*
- *Estos roles irán cambiando de etapa en etapa, manera tal que p. ej. para mover la torre de 3 de 1 a 3, el origen es 1 el destino es 3 y el eje intermediario es 2.*
- *Mencionaremos los ejes como **o**, **d** e **i**.*

Algoritmo de Hanoi

- *El algoritmo de las Torres de Hanoi es el siguiente:*

  Problema: Mover una torre de altura h del eje 1 al eje 3.

- *Se descompone en:*

```
movertorre( h-1, o, i, d )
moverdisco( o, d )
movertorre( h-1, i, d, o )
```

```
#include <stdio.h>
void movertorre( int alt, int or, int de,
  int in );
void moverdisco( int ori, int des );

main() {
  int h, orig = 1, dest = 3, interm = 2;
  printf( "Altura de la Torre: " );
  scanf( "%d", &h );
  printf( "Movimiento de una torre de
  altura " );
  printf( "%2d, de %1d a %1d pasando por
  %1d\n\n", h, orig, dest, interm );
  movertorre( h, orig, dest, interm );
  printf( "\n" );
}
```

```
void movertorre( int alt, int or, int de,
                 int in )
{
  if( alt > 0 ) {
    movertorre( alt - 1, or, in, de );
    moverdisco( or, de );
    movertorre( alt - 1, in, de, or );
  }
}

void moverdisco( int ori, int des )
{
  static long movimiento = 1;
  printf( "Mov. %6ld   de: %1d  a: %1d\n",
  movimiento++, ori, des );
}
```

Estudio de Movimientos

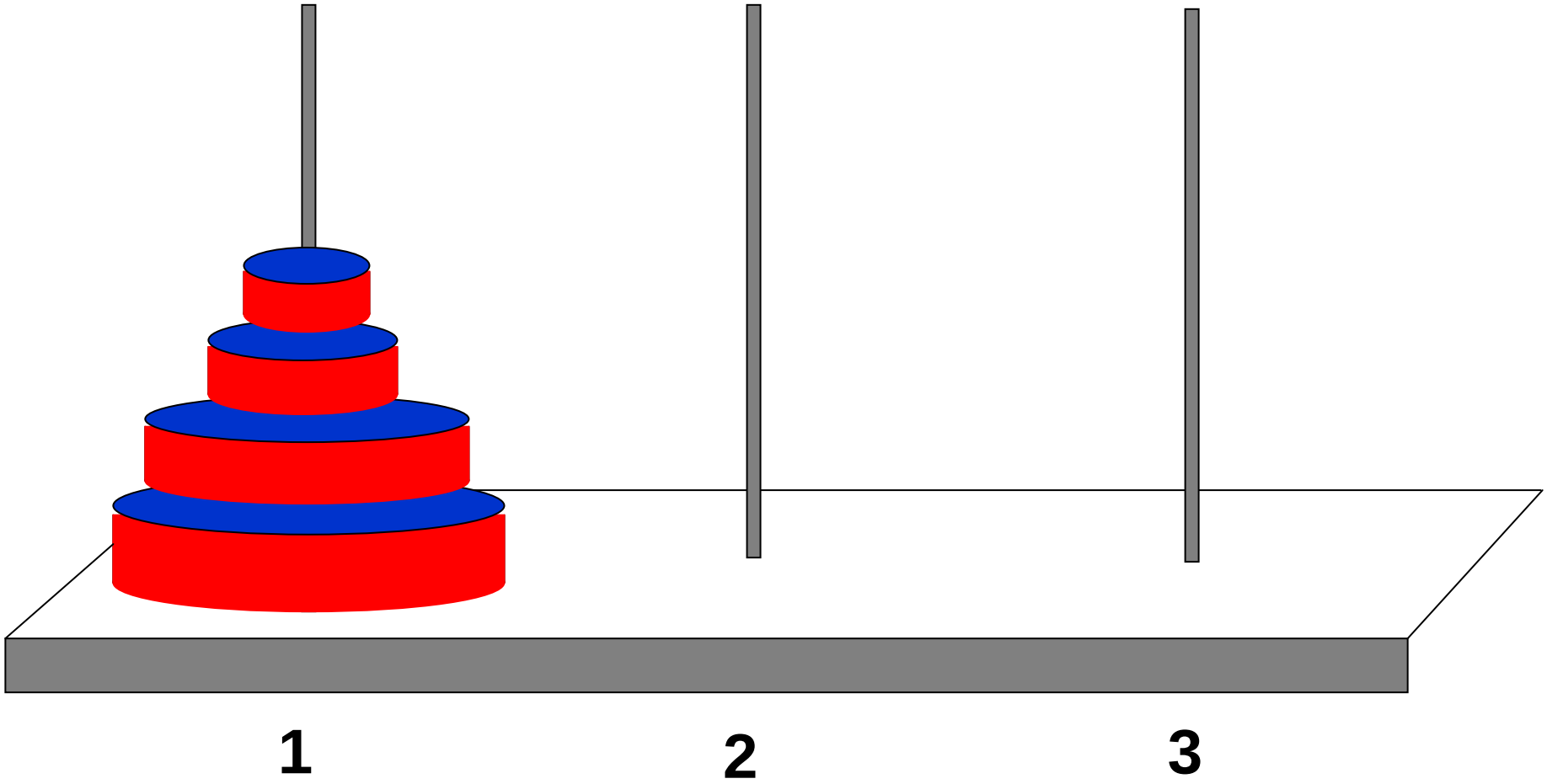
```
movertorre( 4, 1, 2, 3 )
 movertorre( 3, 1, 3, 2 )
   movertorre( 2, 1, 2, 3 )
     movertorre( 1, 1, 3, 2 )
       movertorre( 0, 1, 2, 3 )  SIN EFECTO
       moverdisco( 1, 3 )
       movertorre( 0, 2, 3, 1 )  SIN EFECTO
     moverdisco( 1, 2 )
     movertorre( 1, 3, 2, 1 )
       movertorre( 0, 1, 2, 3 )  SIN EFECTO
       moverdisco( 3, 2 )
       movertorre( 0, 2, 3, 1 )  SIN EFECTO
   moverdisco( 1, 3 )
   movertorre( 2, 2, 3, 1 )
   .....
 moverdisco( 1, 2 )
 movertorre( 3, 3, 2, 1 )
```

Movimiento de la torre de altura 2 desde el eje 2 al eje 3:

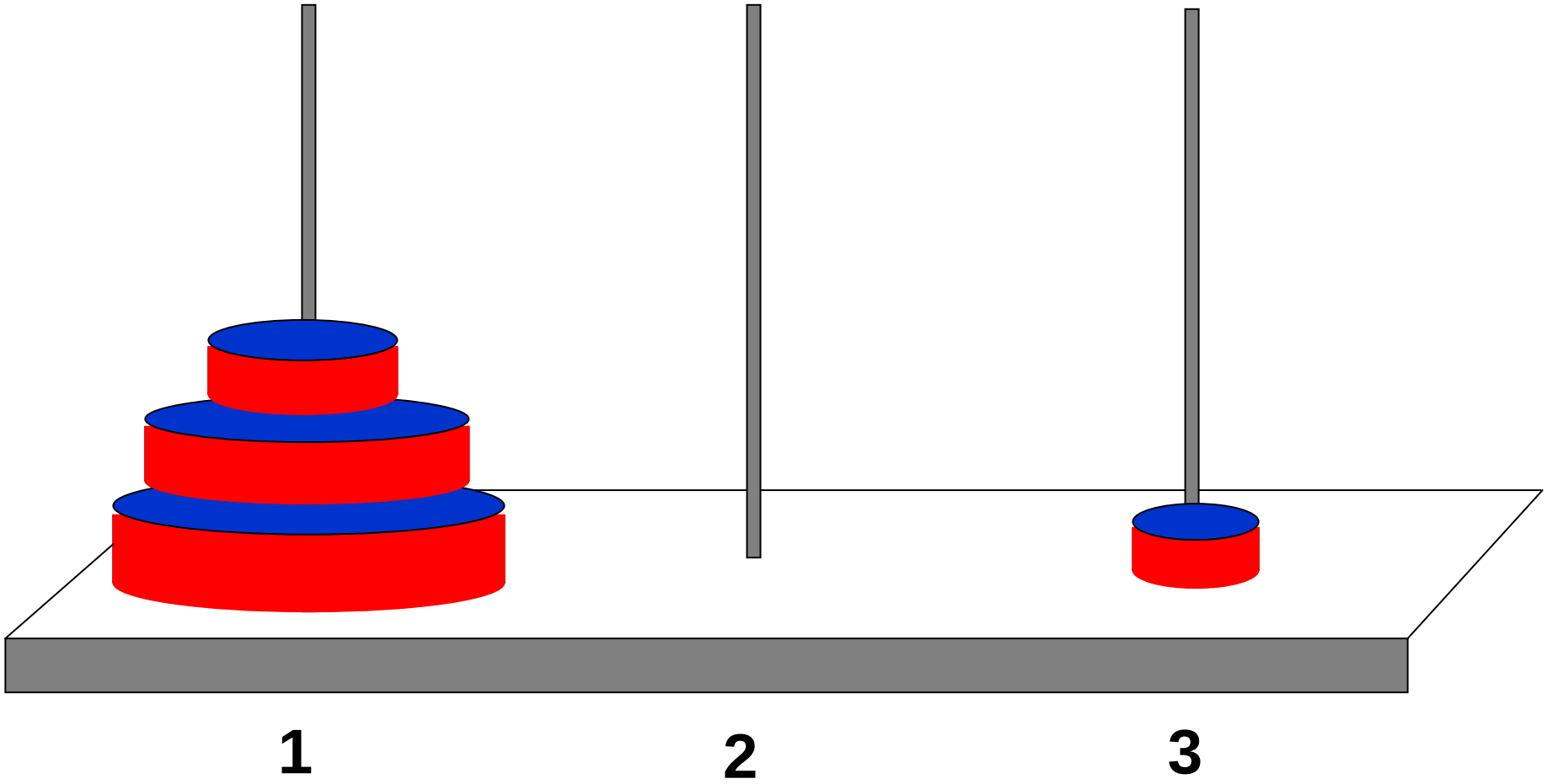
```
movertorre( 2, 2, 3, 1 )
  movertorre( 1, 2, 1, 3 )
    movertorre( 0, 1, 3, 1 )  SIN EFECTO
    moverdisco( 2, 1 )
    movertorre( 0, 3, 2, 1 )  SIN EFECTO
  moverdisco( 2, 3 )
  movertorre( 1, 1, 3, 2 )
    movertorre( 0, 1, 2, 3 )  SIN EFECTO
    moverdisco( 1, 3 )
    movertorre( 0, 2, 3, 1 )  SIN EFECTO
```

Movimiento Completo

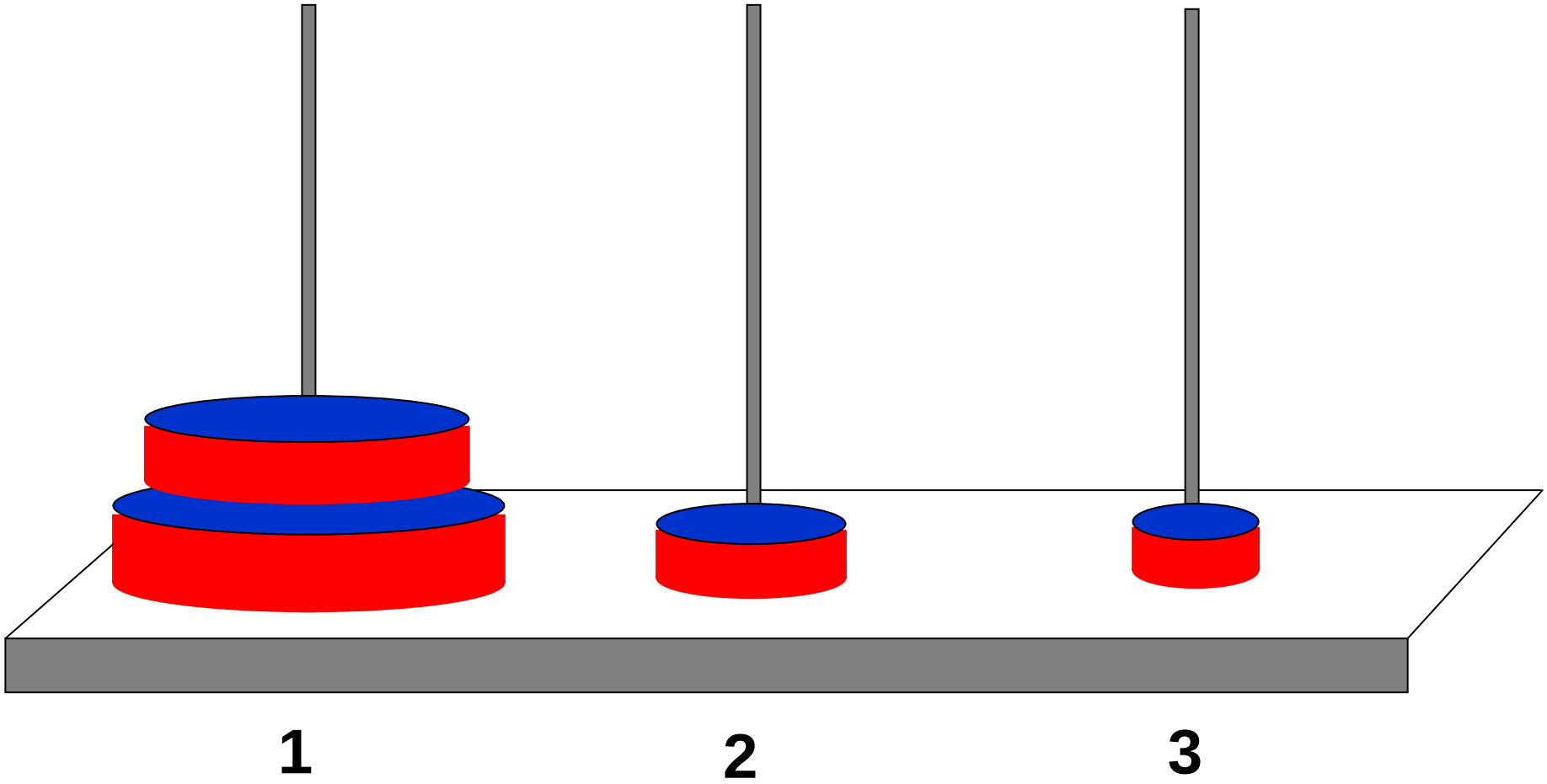
Movimiento completo



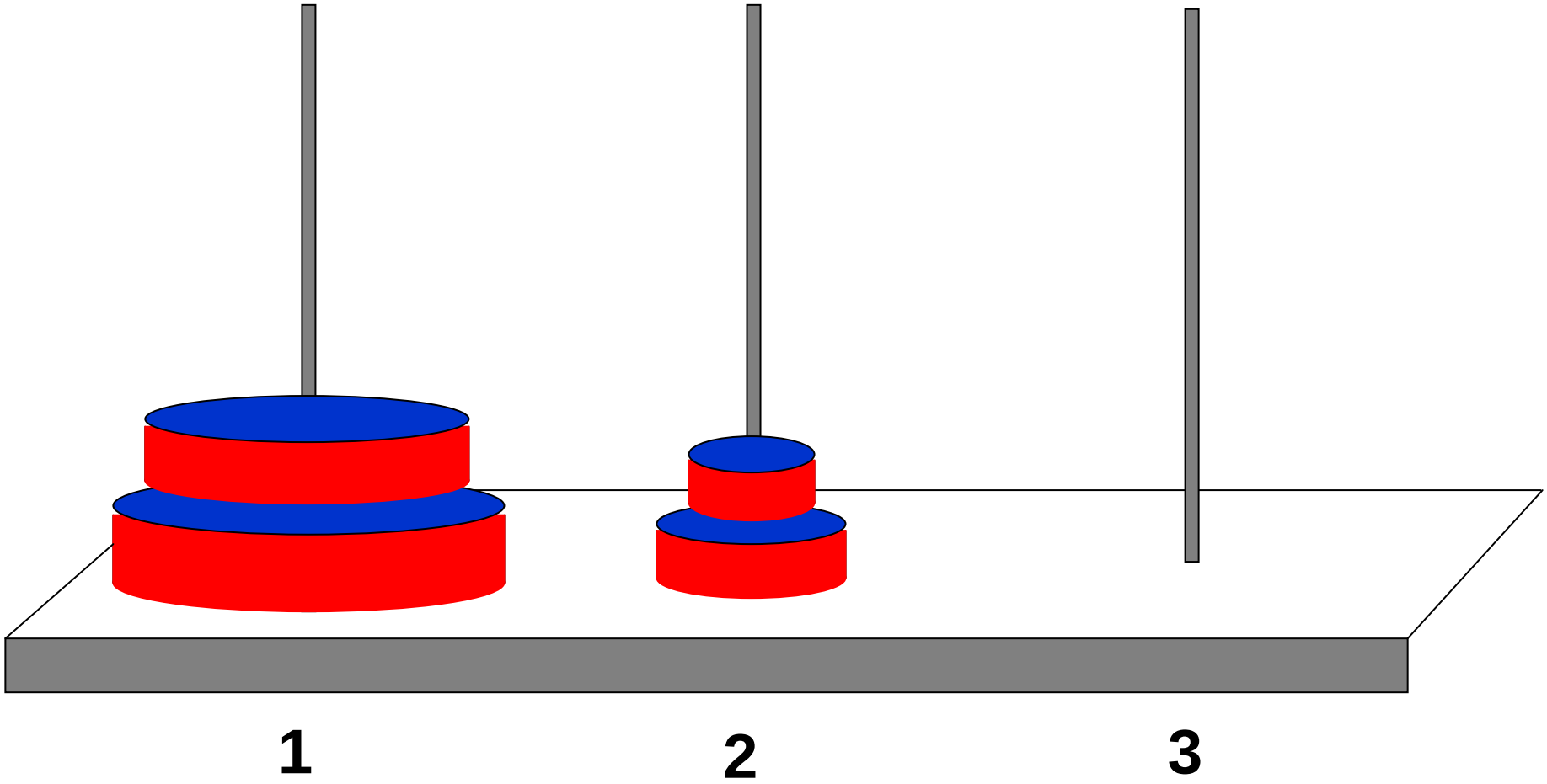
Movimiento completo



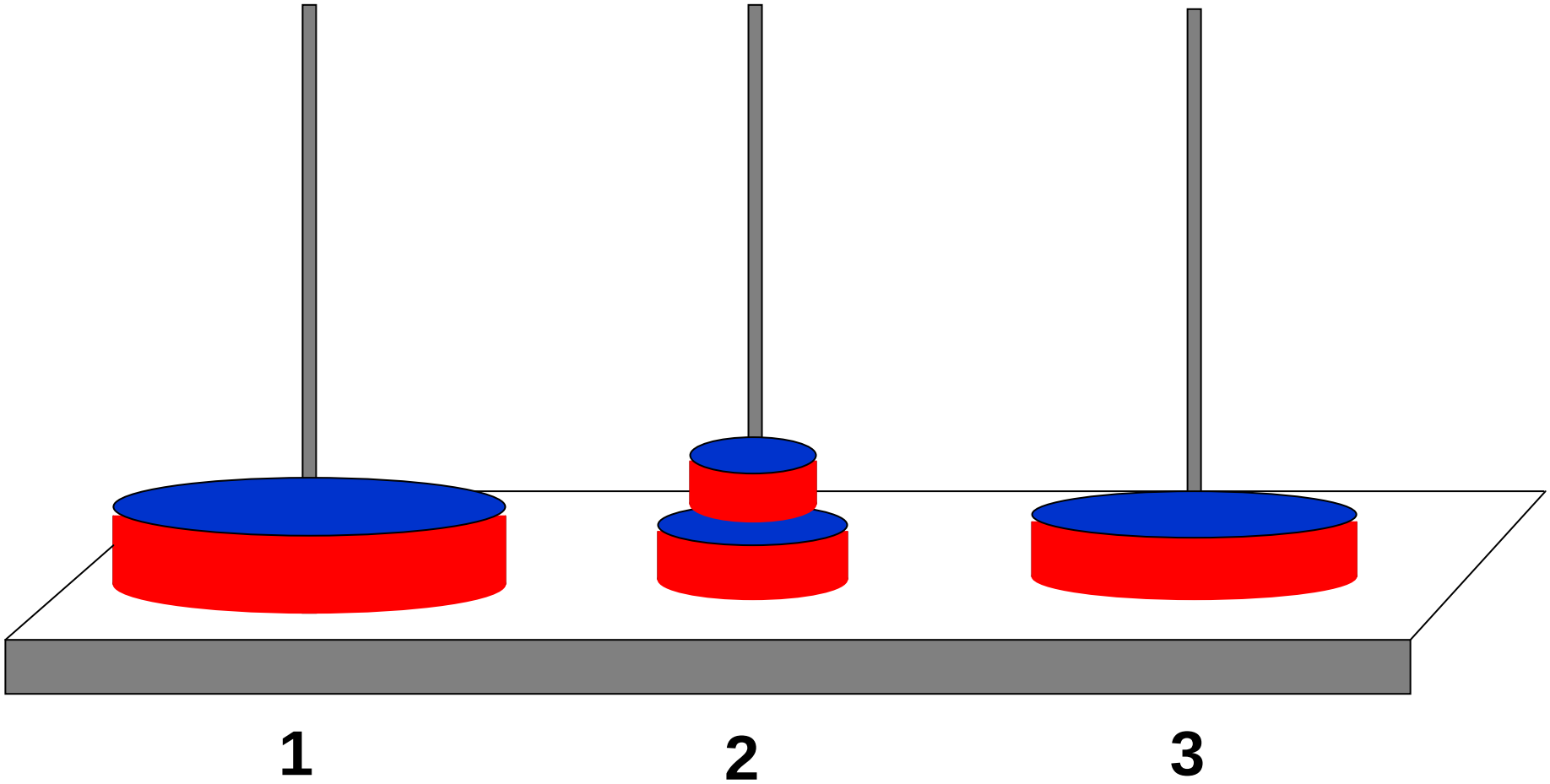
Movimiento completo



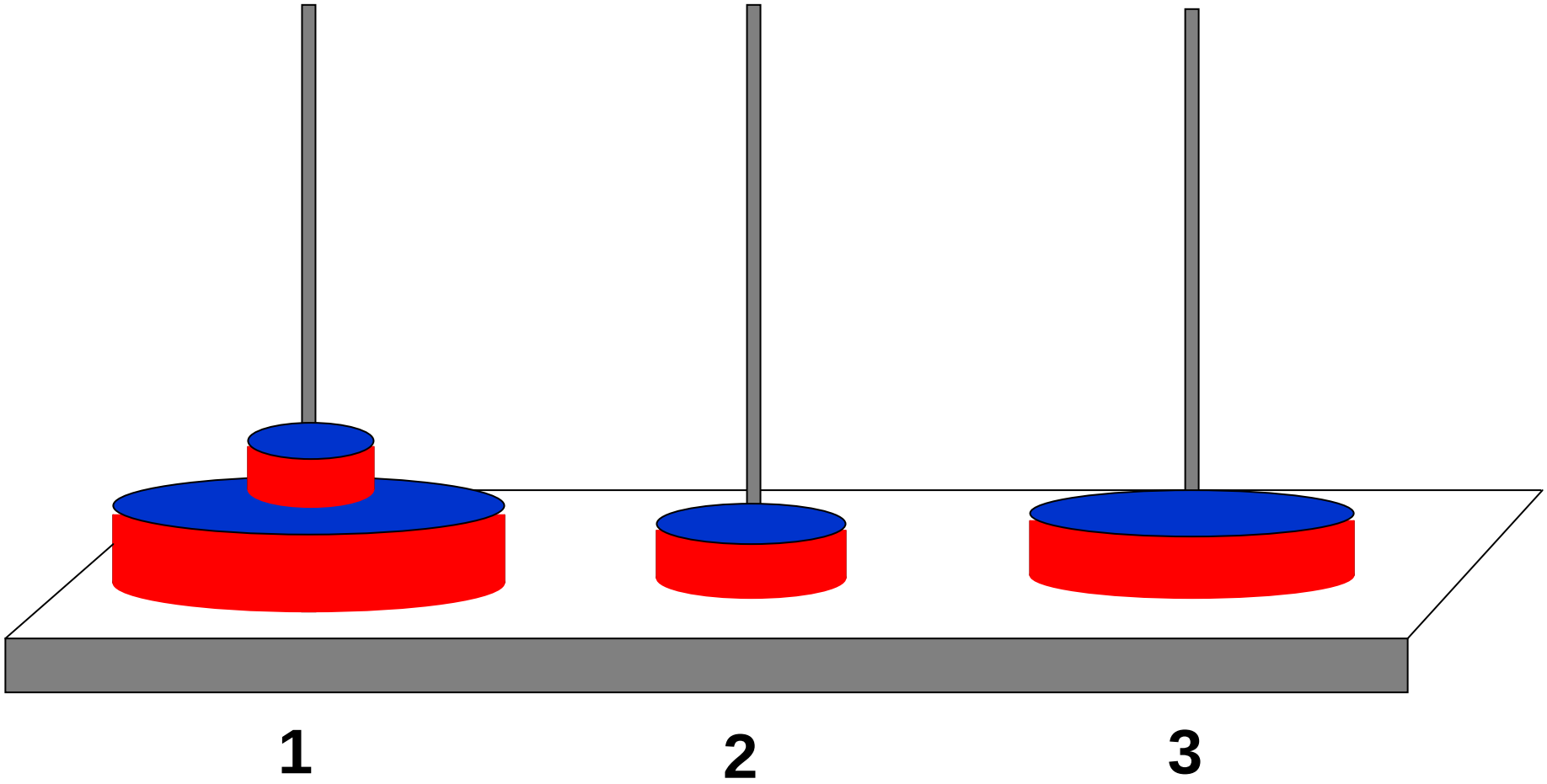
Movimiento completo



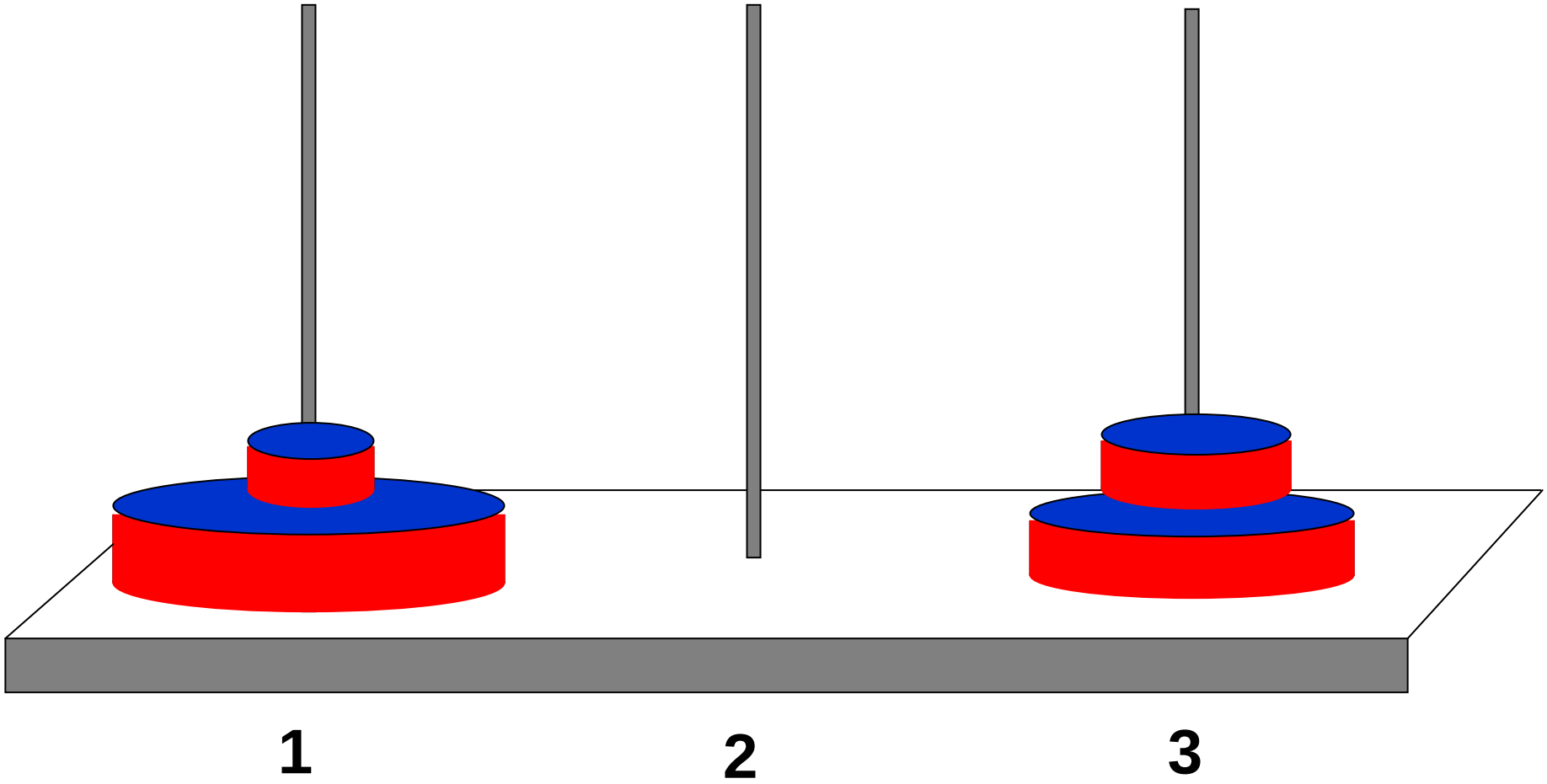
Movimiento completo



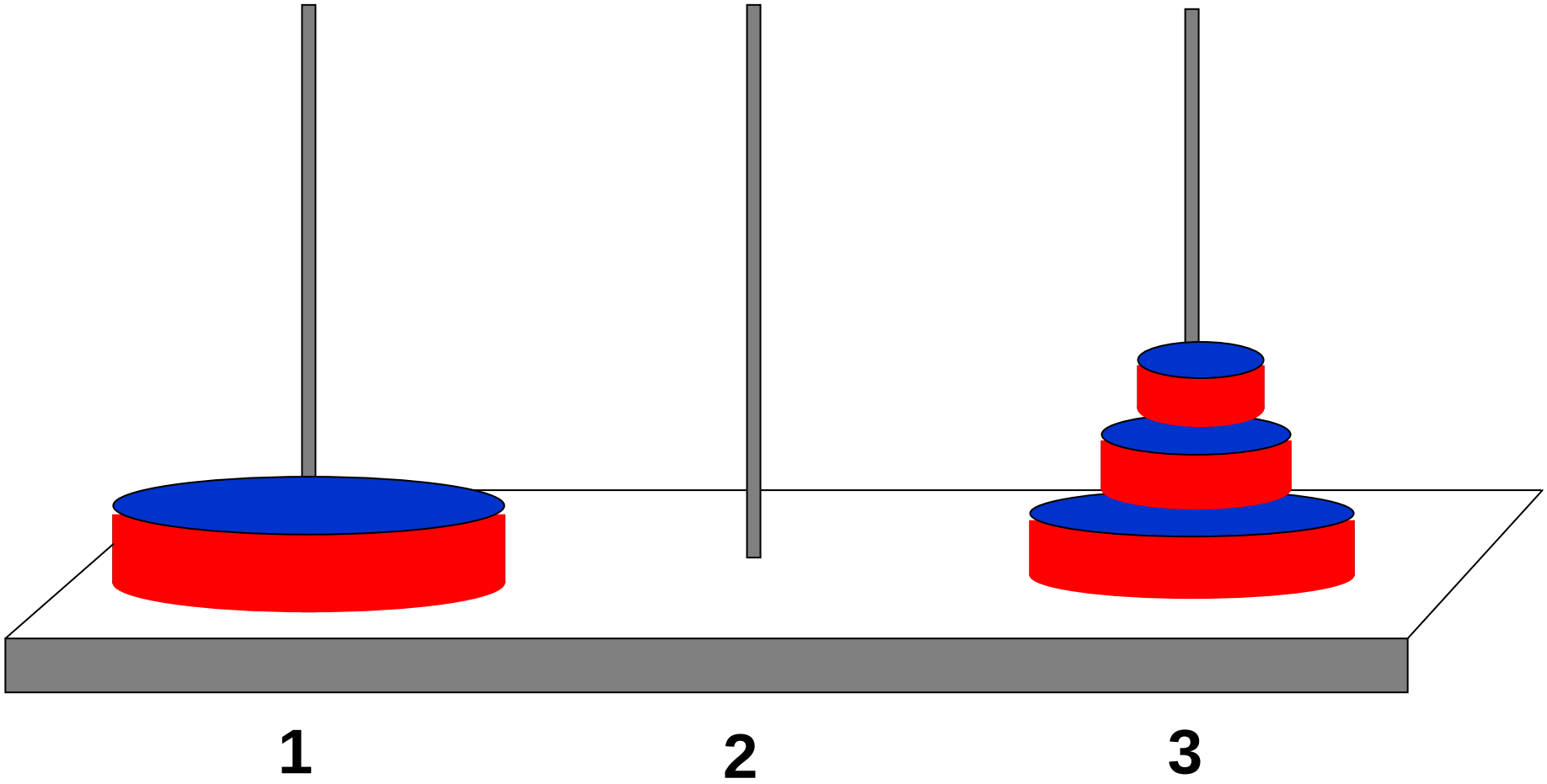
Movimiento completo



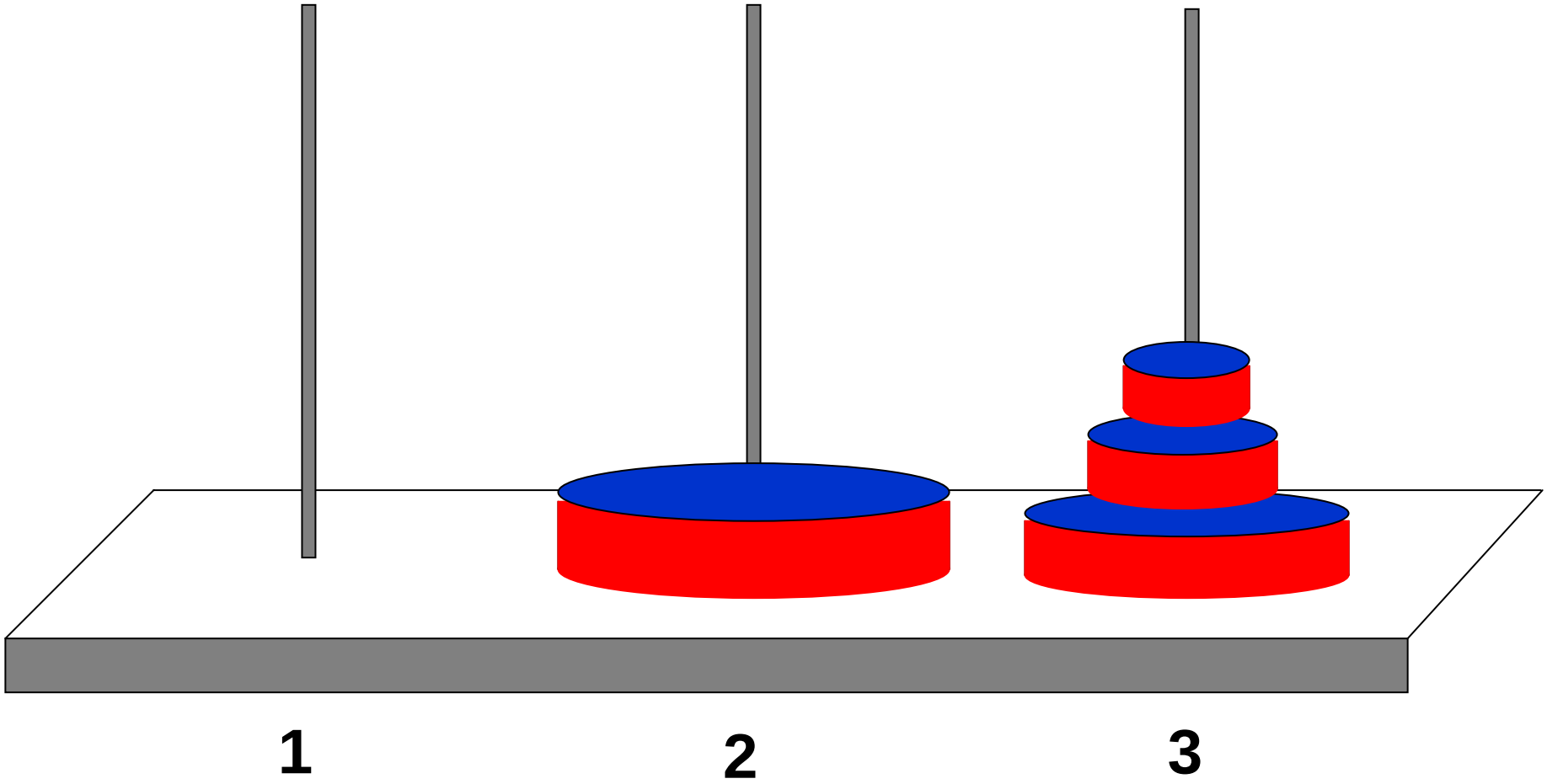
Movimiento completo



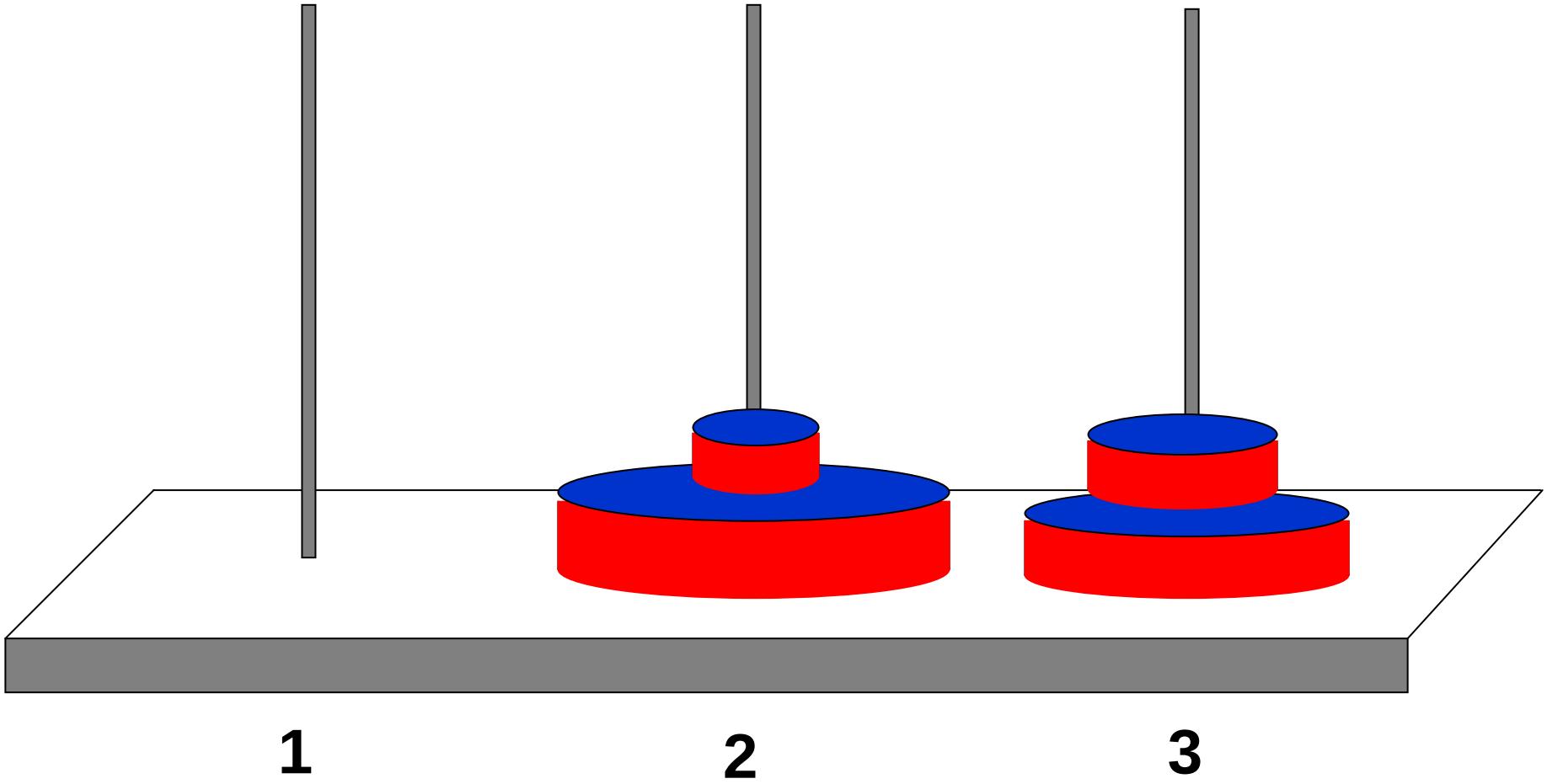
Movimiento completo



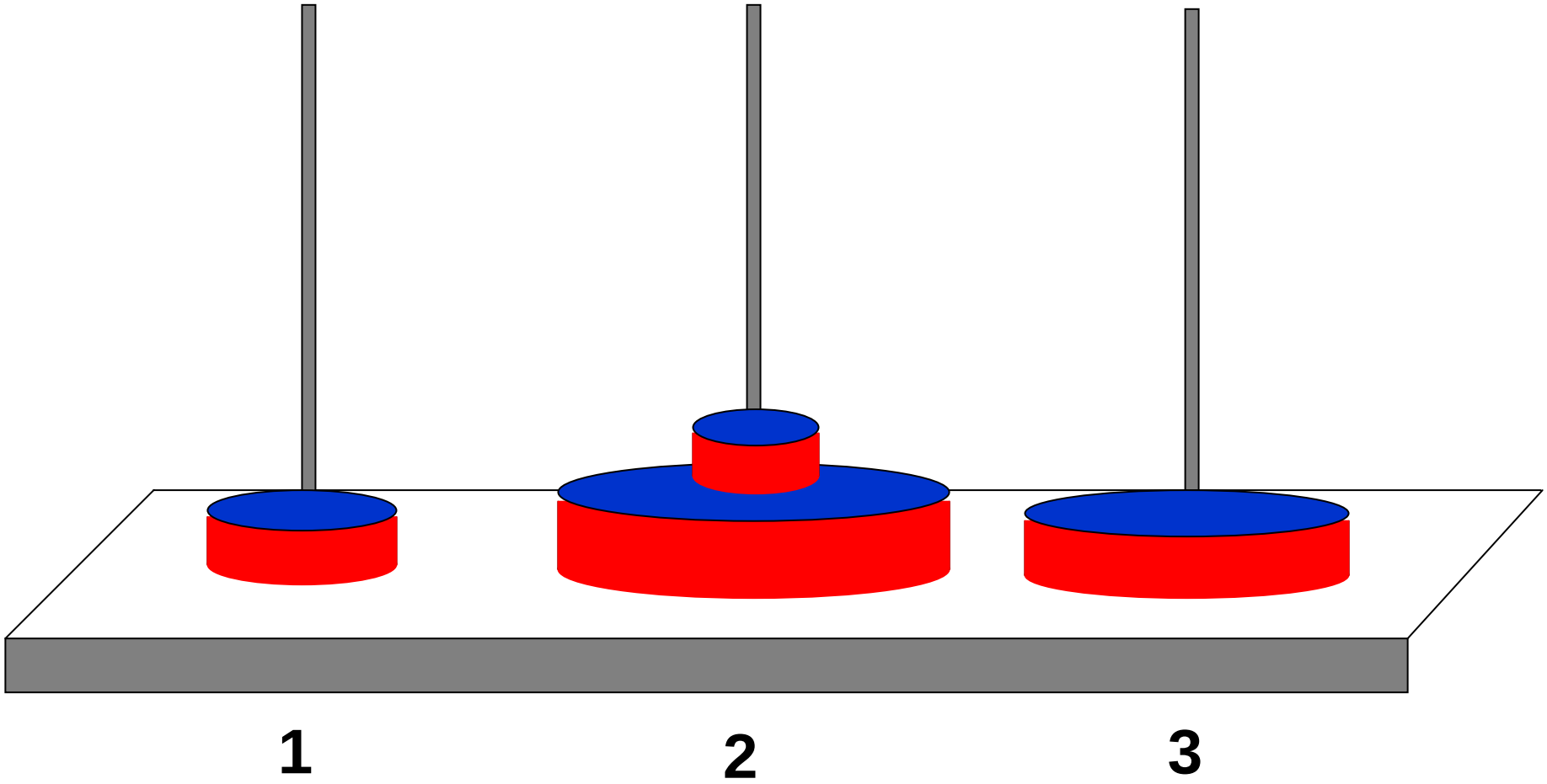
Movimiento completo



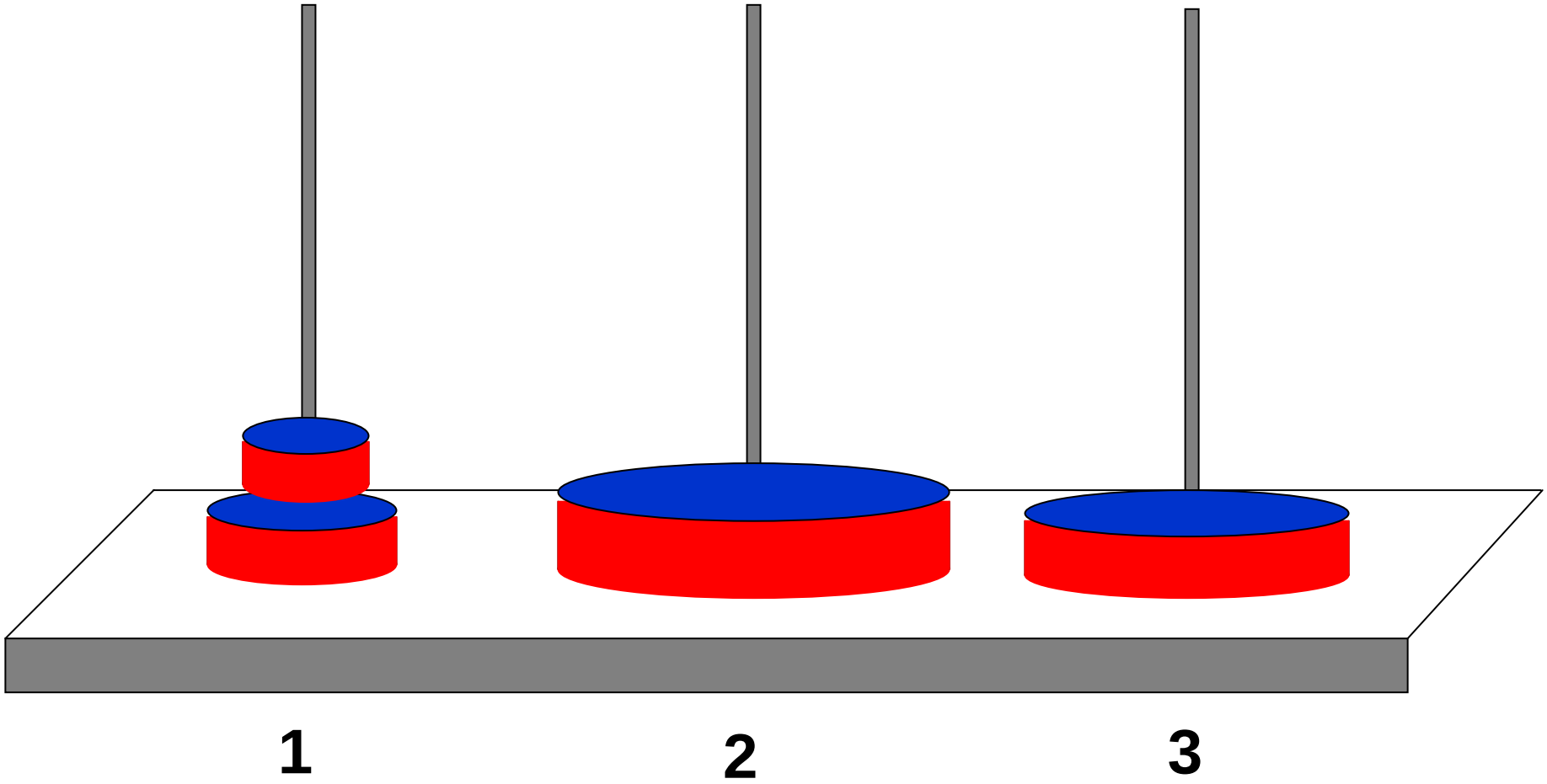
Movimiento completo



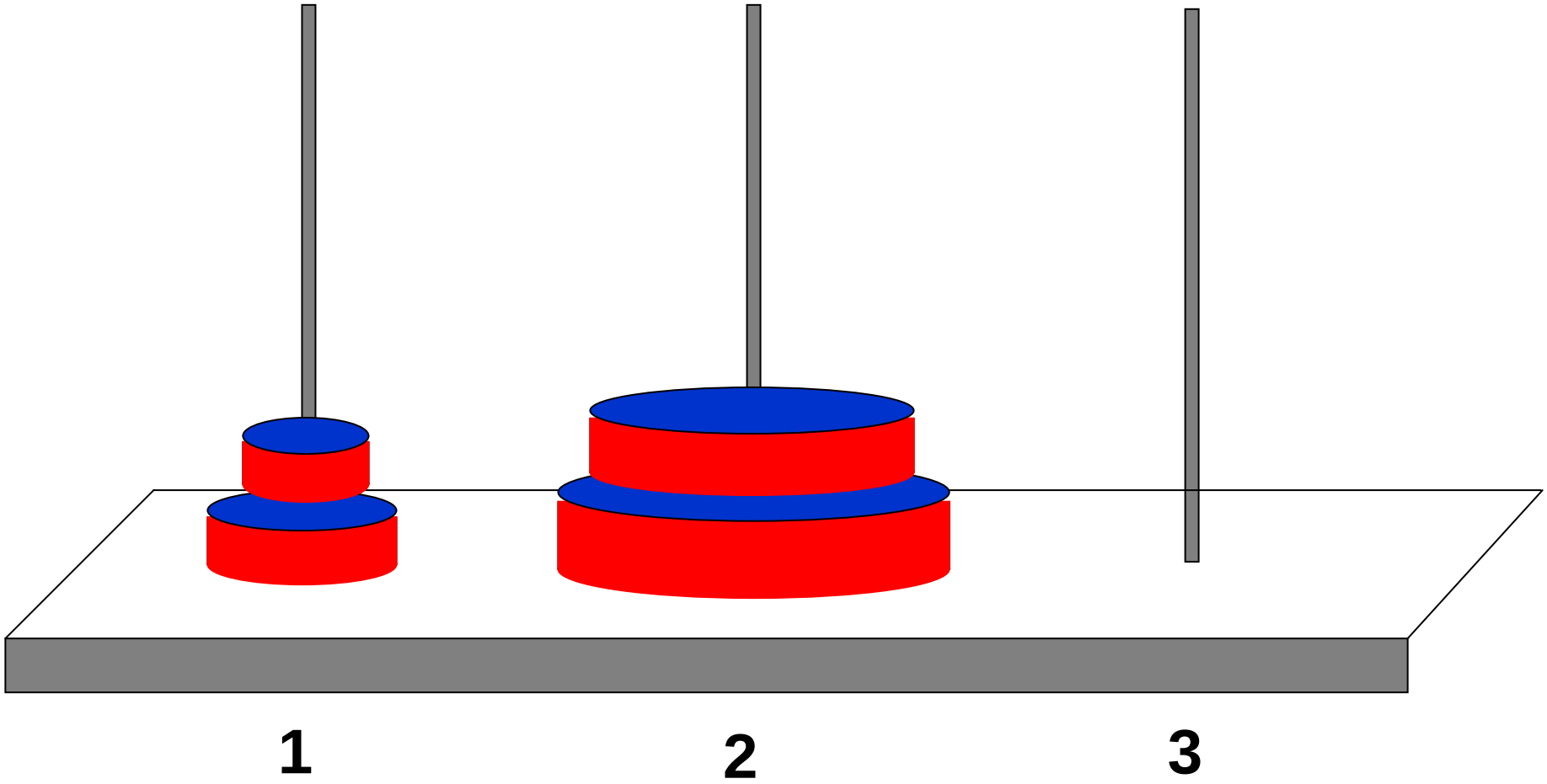
Movimiento completo



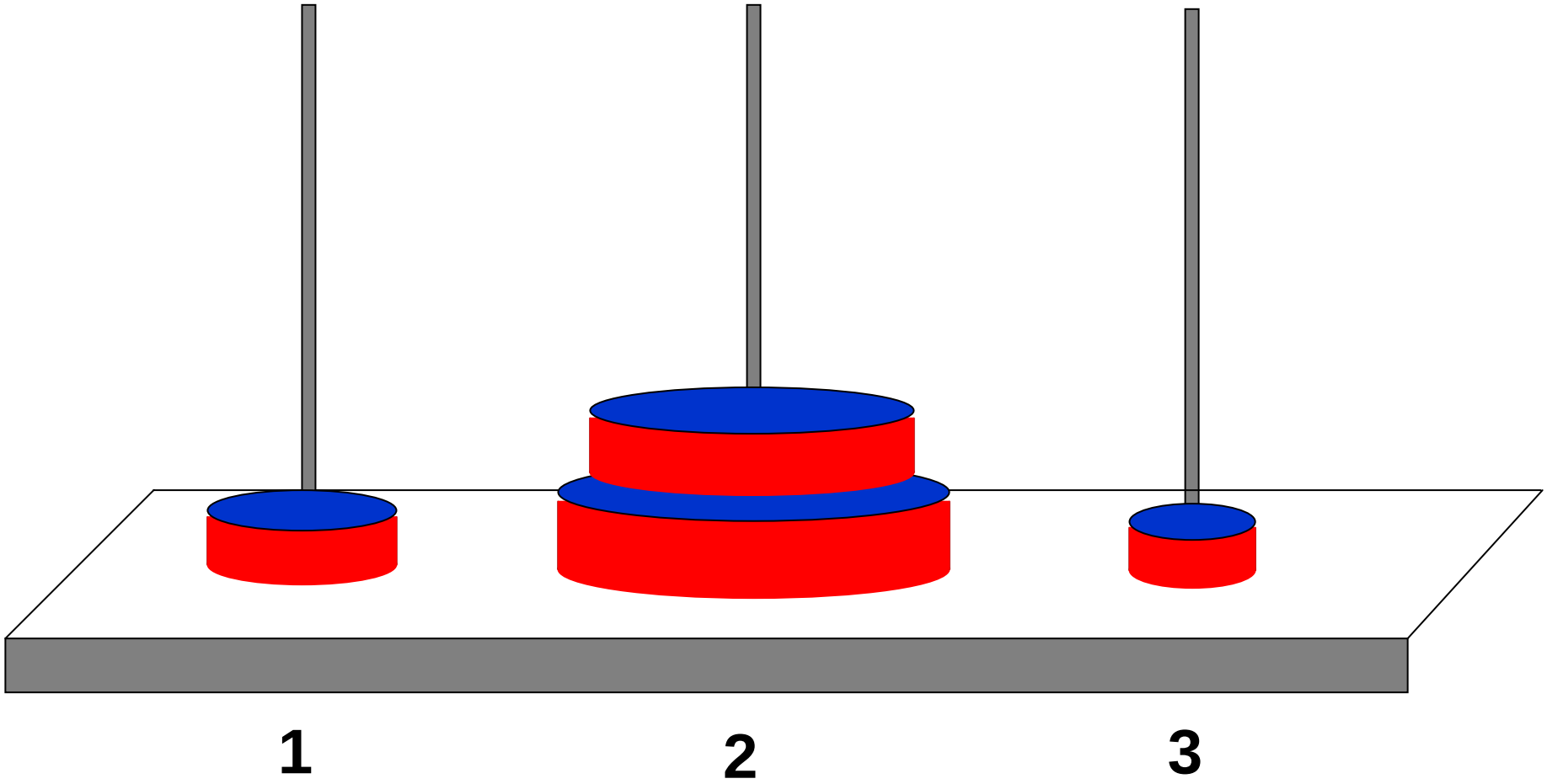
Movimiento completo



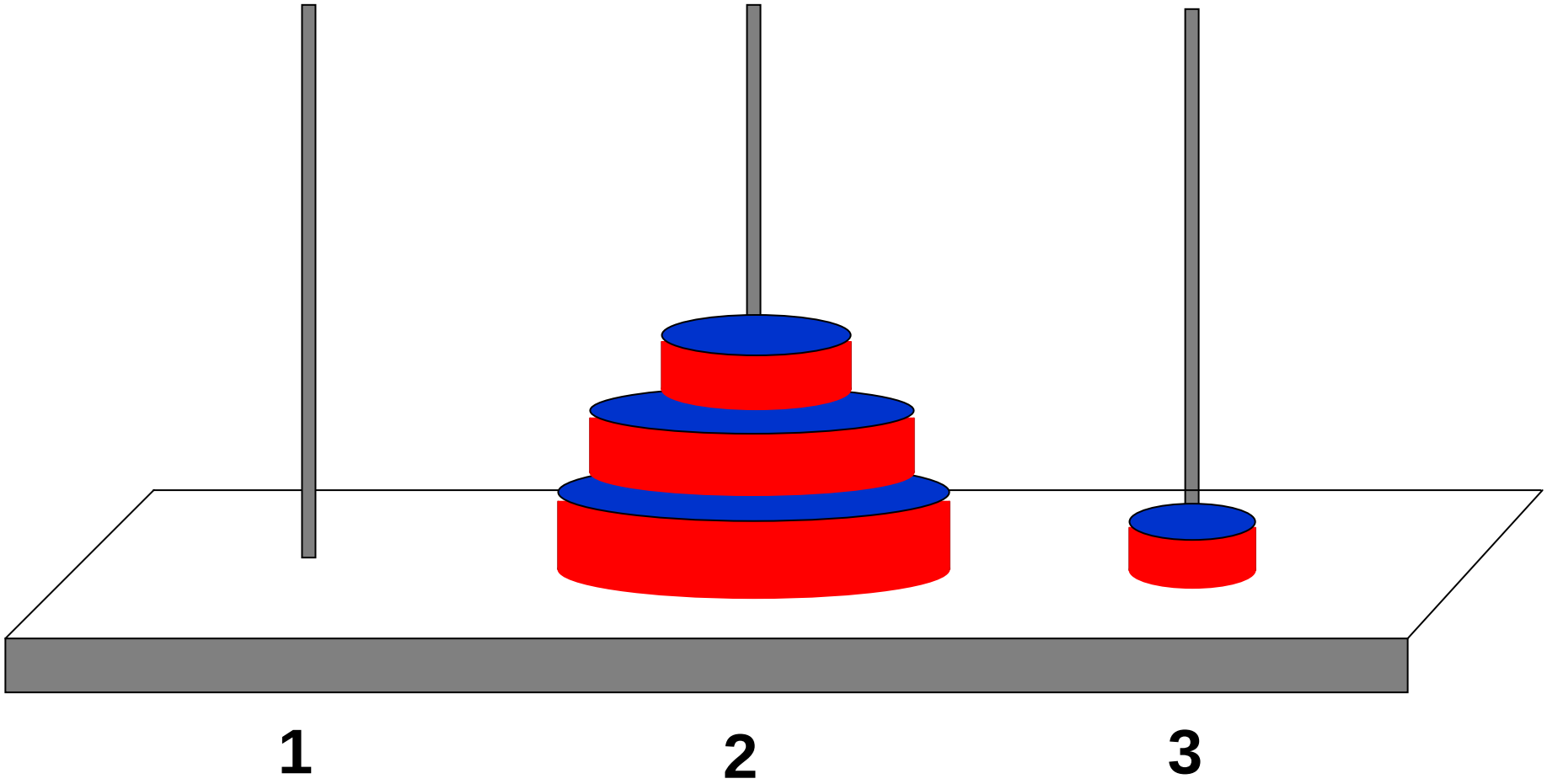
Movimiento completo



Movimiento completo



Movimiento completo



Movimiento completo

